

**KWAME NKRUMAH UNIVERSITY OF SCIENCE AND TECHNOLOGY, KUMASI.
FACULTY OF PHYSICAL AND COMPUTATIONAL SCIENCES
DEPARTMENT OF COMPUTER SCIENCE
FINAL YEAR PROJECT**



PROJECT TOPIC

**ECHOLOCATE: A MOBILE BASED ACTIVE SONAR SYSTEM FOR INDOOR SPATIAL
MAPPING AND 2D FLOOR PLAN GENERATION USING DIGITAL SIGNAL PROCESSING**

SUBMITTED BY:

**FESTUS ABBOAH
ADAMA ABDUL-SAMED
ADOMAKO JOHN KWAKU**

CHAPTER ONE: PROJECT PROPOSAL

1.1 INTRODUCTION AND BACKGROUND

The rapid proliferation of smartphone compute power and embedded sensors has enabled new forms of environmental sensing without specialised hardware. Modern smartphones now carry capable cameras, motion sensors, and audio hardware that can be combined to perceive and map indoor spaces. EchoLocate explores this opportunity by combining multiple sensing modalities on a commodity phone: computer-vision depth sensing through Google's ARCore Depth API, which infers depth from a single moving camera without LiDAR; acoustic sensing using Frequency Modulated Continuous Wave (FMCW) chirps analysed through Digital Signal Processing (DSP); and on-device machine learning through Google ML Kit for detecting doors, obstacles, and signage.

Beyond capturing a single room, EchoLocate allows users to share the floor plans they generate through a crowdsourced community database, so that any user can load a building mapped by someone else and be guided to a destination inside it. In this way EchoLocate functions as an indoor counterpart to outdoor navigation applications — a map of the spaces GPS cannot reach.

The project sits at the intersection of mobile sensing, Digital Signal Processing, computer vision, and Flutter application engineering, with particular emphasis on accessibility for blind and low-vision users, feasibility on commodity hardware, and practical trade-offs under real indoor conditions.

1.2 PROBLEM STATEMENT

Outdoor navigation is a solved problem, but GPS stops working at the entrance of a building. The interiors of most buildings — university campuses, hospitals, malls, and public offices — are not digitally mapped, so people routinely get lost inside unfamiliar spaces. This problem is most severe for blind and low-vision users, who have no reliable way to navigate an unfamiliar interior independently; a white cane detects an obstacle but cannot indicate that a target room lies ahead and to the left. Existing indoor measurement and mapping tools are either expensive and hardware-dependent (laser meters and LiDAR), restricted to a single platform, dependent on the cloud (raising privacy concerns for sensitive indoor layouts), or limited to point measurements with no navigation capability. Furthermore, indoor maps barely exist in the first place because creating them is costly and time-consuming. There is therefore a need for a low-cost, privacy-preserving, cross-platform system that allows ordinary users to generate indoor floor plans and share them, so that anyone — sighted or not — can navigate confidently inside buildings.

1.3 AIM OF THE PROJECT

To design and evaluate EchoLocate, a cross-platform Flutter application that combines computer-vision depth sensing, acoustic Digital Signal Processing, and on-device artificial intelligence to generate crowdsourced indoor floor plans and provide intelligent indoor navigation — including a voice-guided accessibility mode for blind and low-vision users — on commodity smartphones, without relying on LiDAR or dedicated laser hardware.

1.4 SPECIFIC OBJECTIVES

To achieve the aforementioned aim, the following technical objectives will be pursued:

- **Implement Computer-Vision Depth Mapping:** Develop a module using the ARCore Depth API to capture depth-from-motion data on commodity smartphones and accumulate it into a spatial point cloud, without requiring LiDAR or a dedicated depth sensor.
- **Generate 2D Floor Plans:** Implement a line-fitting pipeline (using RANSAC and Hough-transform techniques) that converts the accumulated depth points into clean 2D floor-plan wall segments, complemented by an extruded 3D preview of the captured space.
- **Implement On-Device Semantic Detection:** Integrate Google ML Kit object detection and text recognition to identify and label doors, obstacles, and room/sign text from the live camera feed and place them on the generated floor plan.
- **Repurpose Acoustic DSP for Environment Sensing:** Develop a Dart-native FMCW chirp generation and Fast Fourier Transform (FFT) pipeline that extracts reverberation features to classify the type of space (for example corridor, small room, or large hall) and

to provide a fallback distance estimate when vision-based depth sensing fails in low light or against featureless surfaces.

- **Build a Crowdsourced Map Database:** Implement a community map repository that enables users to upload the floor plans they scan and to browse, load, and navigate maps contributed by other users.
- **Implement Indoor Navigation and Accessibility:** Develop graph-based (A*) turn-by-turn indoor navigation with an augmented-reality waypoint overlay, real-time obstacle alerts, and a fully voice-driven accessibility mode that conveys guidance through audio and haptic feedback.
- **Implement Local-First Persistence and Export:** Implement on-device persistence using the Drift database and a PDF report export module to support offline operation and shareable floor-plan documentation.
- **Evaluate and Document Performance Boundaries:** Conduct controlled tests across a minimum of three indoor environments, measuring floor-plan dimensional accuracy against a tape measure, acoustic room-classification accuracy, and navigation success rate for sighted and accessibility-mode users, while explicitly documenting limitations and failure cases.

1.5 PROJECT SCOPE

The scope of EchoLocate is defined across the following boundaries, balancing technical feasibility with the constraints of a 12-week academic project timeline.

In Scope

- **Cross-platform mobile application:** A Flutter application targeting Android as the primary platform with iOS support, built from a single codebase.
- **Vision-based 2D floor plan generation:** Capture of depth-from-motion data via the ARCore Depth API and conversion of accumulated points into 2D floor-plan wall segments, with an extruded 3D preview of the space.
- **On-device semantic detection:** Real-time detection and labelling of doors, obstacles, and room/sign text using Google ML Kit, processed entirely on the device.
- **Acoustic room classification and fallback sensing:** FMCW chirp emission and FFT-based reverberation analysis to classify room type, and a fallback acoustic distance estimate for situations in which vision-based depth fails. The acoustic DSP component is retained and evaluated as one modality within a multimodal system.
- **Crowdsourced map database:** A community repository allowing users to upload scanned floor plans and to browse, load, and navigate buildings mapped by others, with a basic map-quality indicator.
- **Indoor navigation and accessibility:** Graph-based turn-by-turn navigation with an AR waypoint overlay, real-time obstacle alerts, and a voice-driven accessibility mode using audio and haptic feedback.

- **Local-first storage, sharing, and PDF export:** A Drift database for on-device persistence, cloud-backed map sharing for the community database, and a PDF generation module for shareable floor-plan reports.
- **Prototype-level usability and structured testing:** Core UI workflows (scan, navigate, explore, save/load) and structured benchmarking across three indoor environments.

Out of Scope

- **Full photorealistic 3D reconstruction:** The project is limited to 2D floor-plan generation with a simple extruded 3D preview; dense, real-time photorealistic 3D mesh reconstruction is not included.
- **Outdoor navigation and robustness guarantees:** The system is optimised for indoor environments; outdoor navigation and conditions involving open spaces and high ambient noise are not supported.
- **Fully automatic semantic room understanding:** Detection is limited to the object and text classes provided by ML Kit; complete automatic interpretation of every element in a space is not included.
- **CAD-grade geometric fidelity and direct CAD export:** The system produces reference-grade spatial approximations; architectural-precision outputs and direct AutoCAD/CAD file integration are outside the current scope.
- **Guaranteed sub-centimetre accuracy:** The system targets useful, reference-grade dimensional accuracy rather than the survey-grade precision of dedicated laser scanners.

1.6 MOTIVATION FOR UNDERTAKING THE PROJECT

The primary motivation for undertaking EchoLocate is the opportunity to solve a real, everyday problem — getting lost inside buildings — using only the sensors already present in a standard smartphone. Outdoor navigation has been transformed by mapping applications, yet the moment a person steps inside a building they are on their own. The team is driven by the technical ambition to combine several advanced techniques — computer-vision depth sensing, acoustic signal processing, and on-device machine learning — into a single multimodal sensing system that runs on commodity hardware.

There is also a strong social motivation centred on accessibility. Blind and low-vision individuals face significant difficulty navigating unfamiliar indoor environments independently, and existing tools do not address this need. By providing a voice-guided navigation mode with obstacle alerts and haptic feedback, EchoLocate aims to extend the benefits of indoor mapping to users who need it most. This accessibility focus is underserved globally and especially in Ghana, where assistive indoor-navigation tools are largely unavailable.

Finally, the project is motivated by the absence of indoor maps for most buildings. By making floor-plan creation cheap and crowdsourced — so that sighted users can map a building simply by walking through it — EchoLocate addresses the underlying supply problem and builds a shared, open map of indoor spaces, beginning with the buildings of KNUST. Performing all

sensing and processing on-device further ensures that sensitive indoor layout data remains private to the user.

1.7 JUSTIFICATION OF THE PROJECT

EchoLocate is justified as a low-cost, accessible, and privacy-preserving alternative to the fragmented landscape of existing indoor mapping and navigation tools. No current solution simultaneously provides crowdsourced vision-based floor-plan generation, acoustic environment sensing, indoor navigation, accessibility support for blind and low-vision users, and fully on-device privacy on commodity smartphones — particularly in a developing-country context. By combining the ARCore Depth API for primary spatial mapping with an acoustic DSP pipeline for room classification and fallback sensing, EchoLocate retains a meaningful Digital Signal Processing contribution while delivering reliable, demonstrable results on everyday hardware. Its local-first Drift architecture keeps sensitive floor-plan data on the device, and its crowdsourced model turns ordinary users into map contributors, making indoor maps available for buildings that no commercial provider has mapped.

1.8 PROJECT METHODOLOGY

The development of EchoLocate follows an Agile/Iterative Development Methodology, structured around five weekly-validated phases over a 12-week timeline. Each phase includes measurable acceptance criteria and a documented limitations log to support honest evaluation. This approach manages the technical uncertainty inherent in multimodal sensing by enabling early identification and correction of deficiencies in each sensing layer before full integration. The methodology emphasises continuous benchmarking and documented trade-off decisions over the delivery of a feature-complete product.

The project is structured into five iterative phases, each with a defined acceptance gate:

- **Phase 1 – Research and Foundation:** Conduct a literature review on crowdsourced indoor mapping, ARCore depth-from-motion sensing, and acoustic room classification; set up the Flutter, ARCore, and ML Kit environment; and validate camera/depth capture and FMCW chirp emission on physical devices. Acceptance gate: successful depth capture and chirp emission demonstrated on a physical device.
- **Phase 2 – Computer-Vision Mapping Engine:** Integrate the ARCore Depth API, accumulate depth points into a point cloud, and implement the line-fitting pipeline that converts points into a 2D floor plan; integrate ML Kit object and text detection for semantic labelling. Acceptance gate: a 2D floor plan generated and benchmarked against a tape measure.
- **Phase 3 – Acoustic DSP and Sensor Fusion:** Implement FMCW chirp generation, the FFT pipeline, and reverberation feature extraction; train the room-type classifier in TensorFlow Lite; implement the acoustic fallback for low-light and featureless surfaces; and fuse acoustic, ARCore, and IMU data. Acceptance gate: room-type classification and acoustic fallback operational.
- **Phase 4 – Navigation, Crowdsourcing and Accessibility:** Implement graph-based (A*) routing with turn-by-turn guidance and an AR waypoint overlay; add real-time

obstacle detection with voice and haptic alerts; integrate the Supabase crowdsourced map database, Drift persistence, the 3D preview, the accessibility voice mode, and PDF export. Acceptance gate: a feature-complete application build.

- **Phase 5 – Testing and Refinement:** Conduct map-accuracy testing across three indoor environments, acoustic classification accuracy tests, and navigation success-rate testing for sighted and accessibility-mode users; profile performance and finalise the report. Acceptance gate: validated, deployable build meeting the defined accuracy and usability targets.

Development Tools and Technologies

EchoLocate is built with Flutter (Dart) targeting Android and iOS from a single codebase. The table below lists the development tools and technologies adopted for the project and explains how each will be used.

Tool / Technology	How It Will Be Used
Flutter (Dart)	Primary cross-platform framework; provides a single codebase for the Android and iOS applications, all user-interface screens, and core application logic.
ARCore Depth API	Vision-based depth-from-motion sensing; captures depth data from the moving camera to build the spatial point cloud used for 2D floor-plan generation. Accessed from Flutter through a native Android platform channel.
Google ML Kit	On-device computer vision; performs object detection (doors, obstacles) and text recognition (room numbers and signs) during scanning and navigation, with no cloud processing.
TensorFlow Lite	Runs the trained acoustic room-type classifier model on-device, converting reverberation features into a predicted space type.
Dart FFT Pipeline	Performs the Fast Fourier Transform and reverberation feature extraction on captured audio for room classification and the fallback distance estimate (the project's Digital Signal Processing component).
Drift (SQLite)	Local-first on-device database; stores generated floor plans, scan sessions, and navigation history for fully offline operation.
Supabase	Cloud backend for the crowdsourced map repository; stores and serves community-uploaded floor plans, contributor records, and map-quality ratings.
flutter_tts & speech_to_text	Voice interface; deliver spoken turn-by-turn guidance and accept voice commands for the accessibility mode.
A* Routing Engine (custom Dart)	Computes the shortest navigable path through a loaded floor plan to drive turn-by-turn navigation.
flutter_gl	Renders the extruded 3D preview of the generated floor plan.
pdf (Dart package)	Generates shareable PDF floor-plan reports from captured spatial data.

1.9 PROJECT DELIVERABLES

Upon successful completion of the EchoLocate project within the 12-week development timeline, the following tangible deliverables will be produced and submitted:

Deliverable 1 – Computer-Vision Mapping Engine: A module that integrates the ARCore Depth API to capture depth-from-motion data and accumulate it into a spatial point cloud, together with

a line-fitting pipeline that converts the points into 2D floor-plan wall segments and an extruded 3D preview.

Deliverable 2 – Acoustic DSP and Room-Classification Module: A Dart-native FMCW chirp generation and FFT pipeline with reverberation feature extraction and a TensorFlow Lite room-type classifier, including the fallback distance-estimation routine used when vision-based depth fails. This deliverable retains and demonstrates the Digital Signal Processing contribution of the project.

Deliverable 3 – Indoor Navigation and Routing System: A graph-based A* routing engine providing turn-by-turn guidance, an AR waypoint overlay, and real-time obstacle detection using ML Kit, serving as the analytical core for guiding users through a loaded floor plan.

Deliverable 4 – EchoLocate Mobile Application (Flutter): A fully functional, installable Flutter application for Android and iOS featuring scan, navigate, and explore workflows, a 2D floor-plan view with an extruded 3D preview, project management (create, save, and reload floor plans), and a full accessibility suite comprising a voice-driven mode with audio and haptic feedback for blind and low-vision users.

Deliverable 5 – Crowdsourced Map Database with Local Storage and PDF Export: A Supabase-backed community repository for uploading, browsing, loading, and navigating shared floor plans; a local-first Drift database for on-device persistence; and a PDF generation module that converts captured floor-plan data into shareable documentation.

Deliverable 6 – Evaluation and Test Report: A comprehensive validation report documenting floor-plan dimensional accuracy against a calibrated tape measure across a minimum of three distinct indoor environments (for example a small room, a corridor, and an open hall), acoustic room-classification accuracy, and navigation success rate for sighted and accessibility-mode users, including raw data tables, statistical analysis, and a discussion of limitations.

Deliverable 7 – Final Project Report and Source Code: A complete academic project report adhering to KNUST Department of Computer Science formatting standards, inclusive of all design diagrams, test results, and reflective evaluation. Full, documented source code will be submitted via the designated repository, ensuring reproducibility and academic integrity.

1.10 PROJECT PLAN AND TIMELINES

The EchoLocate project is planned over a 12-week period commencing March 2026, structured around the five iterative development phases defined in the Project Methodology. The subsections below present the Action Plan, Work Breakdown Structure (WBS), Gantt Chart, and key Milestones that together constitute the full project management framework.

Action Plan

The Action Plan below provides a week-by-week breakdown of activities and expected outputs for each phase of the project.

Week	Dates (2026)	Phase	Activity	Expected Output
Week 1	17–23 Mar	Phase 1: Research & Foundation	Conduct literature review on crowdsourced indoor mapping, ARCore depth-from-motion sensing, and acoustic room classification. Research Flutter ARCore and ML Kit plugins.	Annotated bibliography & technology stack decision document
Week 2	24–30 Mar	Phase 1: Research & Foundation	Set up Flutter, ARCore, and ML Kit environment. Validate camera/depth capture and FMCW chirp emission on test devices.	Working depth-capture & chirp prototype; environment setup checklist
Week 3	31 Mar–6 Apr	Phase 2: CV Mapping Engine	Integrate the ARCore Depth API. Accumulate depth-from-motion points into a spatial point cloud.	Depth capture module; raw point cloud on device
Week 4	7–13 Apr	Phase 2: CV Mapping Engine	Implement RANSAC/Hough line-fitting to convert accumulated points into clean 2D floor-plan wall segments.	2D floor plan generation module
Week 5	14–20 Apr	Phase 2: CV Mapping Engine	Integrate ML Kit object and text detection to label doors, obstacles, and room numbers. Benchmark plan dimensions against a tape measure.	Semantic labelling on floor plan; accuracy benchmarking spreadsheet
Week 6	21–27 Apr	Phase 3: Acoustic DSP & Fusion	Implement FMCW chirp generation, the FFT pipeline, and reverberation feature extraction. Train the room-type classifier in TensorFlow Lite.	Acoustic room-classification module
Week 7	28 Apr–4 May	Phase 3: Acoustic DSP & Fusion	Implement the acoustic fallback for low-light/featureless surfaces. Fuse acoustic, ARCore, and IMU data.	Multimodal fusion & fallback subsystem
Week 8	5–11 May	Phase 4: Navigation & Accessibility	Build graph-based A* routing with turn-by-turn guidance and an AR waypoint overlay. Add real-time obstacle detection with voice and haptic alerts.	Navigation engine; AR overlay & obstacle-alert demo
Week 9	12–18 May	Phase 4: Navigation & Accessibility	Integrate the Supabase crowdsourced database, Drift persistence, the 3D preview, the accessibility voice mode, and PDF export.	Feature-complete application build; crowdsource upload & PDF export working
Week 10	19–25 May	Phase 5: Testing & Refinement	Conduct map-accuracy testing across three indoor environments. Run acoustic room-classification accuracy tests.	Accuracy & classification report with raw data
Week 11	26 May–1 Jun	Phase 5: Testing & Refinement	Conduct navigation success-rate testing for sighted and accessibility-mode users. Profile and optimise performance.	Navigation study results; performance profiling report
Week 12	2–8 Jun	Phase 5: Testing & Refinement	Final regression testing, UI polish, and accessibility audit. Complete project report and prepare final code submission.	Final application release build; complete project report

Work Breakdown Structure (WBS)

The Work Breakdown Structure decomposes the entire EchoLocate project into hierarchical, deliverable-oriented tasks. Each task is assigned a unique WBS identifier for traceability across the project schedule.

WBS ID	Task / Deliverable Description	Weeks
1.0	EchoLocate – Multimodal Indoor Mapping & Navigation Application	1–12
1.1	Research and Foundation	1–2
1.1.1	Literature review on crowdsourced mapping, ARCore depth, and acoustic room classification	1
1.1.2	Evaluation and selection of Flutter ARCore, ML Kit, and audio plugins	1
1.1.3	Development environment, ARCore, and ML Kit setup	2
1.1.4	Depth-capture and chirp-emission validation on target device	2
1.2	Computer-Vision Mapping Engine	3–5
1.2.1	ARCore Depth API integration and point-cloud accumulation	3
1.2.2	RANSAC/Hough line-fitting to 2D floor plan	4
1.2.3	ML Kit object and text detection labelling	5
1.2.4	Floor-plan dimensional accuracy benchmarking (vs tape measure)	5
1.3	Acoustic DSP and Sensor Fusion	6–7
1.3.1	FMCW chirp generation and FFT pipeline (Dart)	6
1.3.2	Reverberation feature extraction and room-type classifier (TensorFlow Lite)	6
1.3.3	Acoustic low-light / featureless-surface fallback	7
1.3.4	Multimodal fusion (acoustic + ARCore + IMU)	7
1.4	Navigation, Crowdsourcing and Accessibility	8–9
1.4.1	Graph-based A* routing and turn-by-turn guidance	8
1.4.2	AR waypoint overlay and real-time obstacle detection	8
1.4.3	Supabase crowdsourced upload/browse and map-quality indicator	9
1.4.4	Drift persistence, 3D preview, and PDF export	9
1.4.5	Accessibility voice mode (voice + haptic feedback)	9
1.5	Testing and Refinement	10–12
1.5.1	Map-accuracy testing across three indoor environments	10
1.5.2	Acoustic classification accuracy and statistical analysis	10
1.5.3	Navigation success-rate study (sighted vs accessibility)	11
1.5.4	Performance profiling and optimisation	11
1.5.5	Final regression testing, UI polish, and accessibility audit	12
1.5.6	Project report compilation and source-code submission	12

Gantt Chart

The Gantt Chart below maps each phase and its associated tasks against the 12-week project timeline (March – June 2026). Shaded cells indicate periods of active work.

Task / Phase	W1	W2	W3	W4	W5	W6	W7	W8	W9	W10	W11	W12
Phase 1: Research & Foundation												
1.1 Literature review & tech stack												
1.2 ARCore/ML Kit setup & depth+chirp validation												
Phase 2: CV Mapping Engine												
2.1 ARCore depth integration & point cloud												
2.2 Line-fitting to 2D floor plan												
2.3 ML Kit labelling & accuracy benchmark												
Phase 3: Acoustic DSP & Fusion												
3.1 FMCW + FFT + room classifier												
3.2 Fallback & multimodal fusion												
Phase 4: Navigation, Crowdsourcing & Accessibility												
4.1 Routing, AR overlay & obstacle detection												
4.2 Crowdsourcing, persistence, 3D, accessibility, PDF												
Phase 5: Testing & Refinement												
5.1 Map accuracy & classification testing												
5.2 Navigation study & profiling												
5.3 Final testing & report												

Milestones — Major Targets and Achievements

The following five milestones represent the critical checkpoints against which project progress will be evaluated. Each milestone corresponds to a phase completion gate and will be reviewed by the team and supervising lecturer before proceeding to the next phase.

M1	Project Foundation Complete	Target Date: End of Week 2 (30 Mar 2026)
-----------	------------------------------------	---

All foundational research on crowdsourced indoor mapping, ARCore depth-from-motion sensing, and acoustic room classification is completed and documented. The Flutter, ARCore, and ML Kit environment is fully configured, and a proof-of-concept demonstrating successful camera/depth capture and FMCW chirp emission on a physical device is validated.	Evidence / Outputs: • Annotated literature review document • Environment setup checklist • Depth-capture & chirp demo build
--	---

M2	Computer-Vision Mapping Engine Operational	Target Date: End of Week 5 (20 Apr 2026)
The vision-based mapping engine is operational: ARCore depth data is accumulated and converted into a 2D floor plan, with ML Kit object and text labelling. Floor-plan dimensions are benchmarked against a calibrated tape measure in a controlled indoor environment.		Evidence / Outputs: • ARCore depth & point-cloud module • 2D floor-plan generation with labelling • Dimensional accuracy benchmarking report

M3	Multimodal Sensing Complete	Target Date: End of Week 7 (4 May 2026)
The acoustic DSP component is operational: FMCW chirp generation, the FFT pipeline, and the TensorFlow Lite room-type classifier are implemented, and the acoustic fallback is fused with ARCore and IMU data to support sensing where vision fails.		Evidence / Outputs: • FMCW + FFT module • Room-type classifier • Fallback & multimodal fusion subsystem

M4	Feature-Complete Application	Target Date: End of Week 9 (18 May 2026)
EchoLocate is feature-complete. Graph-based navigation with turn-by-turn guidance, the AR waypoint overlay, real-time obstacle detection, the Supabase crowdsourced database, Drift persistence, the 3D preview, PDF export, and the full accessibility voice mode are all implemented and integrated.		Evidence / Outputs: • Fully integrated Flutter build (Android & iOS) • Navigation & crowdsource demo • Accessibility voice-mode demo • PDF export working

M5	Validated and Deployable Product	Target Date: End of Week 12 (8 Jun 2026)
EchoLocate has passed map-accuracy tests across a minimum of three indoor environments, acoustic classification accuracy tests, and navigation success-rate testing for sighted and accessibility-mode users. Performance profiling confirms acceptable utilisation on mid-range devices, all identified bugs are resolved, and the final report and source code are submitted.		Evidence / Outputs: • Evaluation report (3 environments) • Final release build • Performance profiling report • Complete project report & source code

CHAPTER TWO: REVIEW OF RELATED SYSTEMS

2.1 INTRODUCTION

This chapter reviews existing systems that are directly related to the core functionality of EchoLocate: mobile-based indoor spatial mapping, 2D floor plan generation, indoor navigation, and accessible spatial awareness. The systems examined represent both commercial and research-grade tools that address overlapping problem domains. For each system, key features, good attributes, and notable limitations are identified and discussed. The insights derived from this review informed the design decisions and technical trade-offs adopted in the proposed EchoLocate system. A comparative analysis table and a justification for the proposed system are also presented.

2.2 OVERVIEW OF EXISTING SYSTEMS

The challenge of accurate indoor spatial measurement, floor plan generation, and indoor navigation has attracted diverse technological approaches across academia and industry. Conventional indoor mapping once required dedicated surveying equipment such as total stations and laser rangefinders, making the process expensive, time-consuming, and inaccessible to the average user (Gu et al., 2015). The proliferation of smartphones embedded with high-performance sensors — including accelerometers, gyroscopes, magnetometers, depth-capable cameras, and high-quality microphones — has opened new avenues for consumer-grade spatial sensing (Mirowski et al., 2013).

Contemporary solutions can be broadly categorised into: (i) Augmented Reality (AR)- and computer-vision-based measurement tools that rely on cameras and visual-inertial odometry; (ii) dedicated laser and LiDAR instruments that offer professional-grade precision; (iii) acoustic sensing systems that exploit the physics of sound propagation; and (iv) hybrid approaches that combine multiple sensing modalities to improve reliability and coverage (Faggion et al., 2020). Despite this variety, almost all available tools stop at measurement or floor-plan capture; very few extend to indoor navigation, and none combine community-contributed maps with accessibility support for blind and low-vision users.

The following sections examine four representative systems that collectively illustrate both the state of the art in indoor spatial mapping and the unmet needs — navigation, crowdsourcing, accessibility, and on-device privacy — that motivate EchoLocate.

2.3 REVIEW OF EXISTING SYSTEMS

A. MagicPlan

MagicPlan is a commercial mobile application developed by Sensopia Inc. that enables users to capture room dimensions and generate 2D floor plans on smartphones and tablets (Sensopia, 2023). The application originally relied on the gyroscope and camera for room corner detection, and has since integrated ARKit and ARCore support, as well as dedicated LiDAR scanning on newer Apple devices. MagicPlan produces exportable floor plans in DXF, PDF, and JPG formats and is widely adopted in real estate, insurance assessment, and interior design (Prattico et al., 2021).

- AR-assisted room scanning using the device camera and gyroscope.
- LiDAR integration on compatible Apple devices for enhanced accuracy.
- Automatic 2D and 3D floor plan generation.
- Export to industry-standard formats including DXF, PDF, and JPG.
- Cloud project storage and team collaboration features.

Good Features of the System (MagicPlan):

- Supports both manual and AR-assisted measurement workflows, catering to a range of skill levels and devices (Prattico et al., 2021).
- Generates professional-quality 2D and 3D floor plans automatically, eliminating the need for manual drafting.
- Exports plans in DXF, enabling direct compatibility with CAD tools such as AutoCAD (Sensopia, 2023).
- Cross-platform availability on iOS and Android broadens accessibility.

Limitations of the System (MagicPlan):

- The free tier severely restricts export and project count; professional features require a paid subscription, which may be cost-prohibitive in developing economies (Sensopia, 2023).
- High accuracy depends on LiDAR hardware available only on premium Apple devices, creating a hardware barrier for Android and mid-range users.
- Requires cloud connectivity for synchronisation and export, raising privacy concerns for sensitive floor-plan data (Gu et al., 2015).
- Provides no indoor navigation, no community-contributed maps, and no accessibility features for blind or low-vision users.

B. Google Measure / AR Ruler Apps

Google Measure was a mobile application developed by Google LLC that leveraged the ARCore platform to provide real-time augmented-reality distance and area measurements on Android devices (Google LLC, 2021). It used visual-inertial odometry to overlay measurement data onto the live camera feed. Although Google discontinued the standalone app in 2021, its functionality persists in numerous AR Ruler applications built on ARCore and ARKit (Faggion et al., 2020), representing an important class of AR-based mobile measurement systems relevant to EchoLocate.

- Real-time AR distance and area measurement via the device camera.
- Point-and-tap measurement interface requiring no physical contact with surfaces.
- Instant on-screen display of measurements overlaid on the camera view.
- No additional hardware beyond a standard ARCore-compatible device.

Good Features of the System (Google Measure / AR Ruler Apps):

- Intuitive, touchscreen-driven interface requiring no technical expertise (Google LLC, 2021).
- Operates on standard smartphone hardware without dedicated sensors, making it broadly accessible (Mirowski et al., 2013).
- Real-time AR overlay provides immediate visual feedback (Faggion et al., 2020).
- Built on the actively maintained ARCore framework, ensuring broad device compatibility — the same foundation EchoLocate builds upon for vision-based depth sensing.

Limitations of the System (Google Measure / AR Ruler Apps):

- Accuracy is highly sensitive to lighting; poor illumination degrades visual feature tracking and introduces errors (Google LLC, 2021).
- Cannot generate complete floor plans; supports only point-to-point or area measurements (Faggion et al., 2020).
- AR-based tracking is susceptible to drift over longer distances, causing cumulative errors (Prattico et al., 2021).
- Provides no navigation, no shared maps, and no accessibility support.

C. Leica BLK360 Imaging Laser Scanner

The Leica BLK360 is a professional-grade imaging laser scanner developed by Leica Geosystems for high-accuracy 3D indoor and outdoor spatial capture (Leica Geosystems, 2022). It emits pulsed laser beams across a full 360-degree field of view to build a dense 3D point cloud at millimetre-level accuracy, integrating panoramic and thermal imaging, and pairs with Leica Register 360 for point-cloud processing. The BLK360 represents the gold standard in indoor laser scanning and serves as a benchmark for accuracy comparison (Gu et al., 2015).

- Full 360-degree laser scanning at millimetre-level accuracy.
- Acquisition of up to 360,000 points per second.
- Integrated panoramic HDR photography and thermal imaging.
- Compatibility with Register 360, AutoCAD, and Revit for professional processing.

Good Features of the System (Leica BLK360):

- Delivers millimetre-range accuracy across complex environments, far exceeding smartphone tools (Leica Geosystems, 2022).
- Full 360-degree scanning eliminates blind spots and produces a comprehensive 3D model in one session (Gu et al., 2015).
- Outputs industry-standard point-cloud data for CAD and BIM workflows.

Limitations of the System (Leica BLK360):

- Extremely high cost (£14,000–18,000 per unit) makes it inaccessible to individual users, students, and small-scale professionals — directly motivating affordable alternatives like EchoLocate (Leica Geosystems, 2022).
- Requires dedicated post-processing software and technical expertise (Gu et al., 2015).

- No smartphone integration; it is a standalone hardware instrument and cannot perform on-the-move mapping.
- Offers no navigation, no community maps, and no accessibility features.

D. RoomScan Pro

RoomScan Pro is an iOS application developed by Locometric Ltd. that uses the iPhone's microphone and compass to scan room boundaries and generate 2D floor plans (Locometric, 2022). The user holds the phone against each wall; the microphone detects the acoustic change at contact and the compass records orientation, and the measurements are assembled into a closed 2D floor plan. As the reviewed system that most closely shares EchoLocate's use of smartphone audio hardware, it is a primary point of technical comparison (Mirowski et al., 2013).

- Acoustic contact detection using the device microphone to identify wall positions.
- Compass-driven orientation recording for wall angle measurement.
- Automatic 2D floor plan assembly from sequential wall measurements.
- Export to PDF and DXF.

Good Features of the System (RoomScan Pro):

- Demonstrates that smartphone audio hardware can support meaningful spatial sensing — a premise EchoLocate extends through acoustic room classification (Mirowski et al., 2013).
- Performs all processing on-device with no cloud dependency, ensuring privacy and offline use (Locometric, 2022).
- Produces closed, geometrically consistent 2D floor plans exportable to DXF (Locometric, 2022).

Limitations of the System (RoomScan Pro):

- Exclusive to iOS; unavailable on Android, the dominant global platform, limiting accessibility in developing markets (Locometric, 2022).
- The contact method requires the user to physically touch the phone to every wall, which is impractical around obstacles and for elevated surfaces (Mirowski et al., 2013).
- Provides no indoor navigation and no community-contributed maps.
- Cannot support assistive use cases for visually impaired users, as it requires precise physical alignment with surfaces and provides no audio or haptic spatial feedback (Mirowski et al., 2013).

2.4 COMPARATIVE SYSTEM ANALYSIS

The table below presents a comparative overview of the four reviewed systems evaluated against key functional criteria relevant to EchoLocate's design goals. The "Proposed System" column reflects EchoLocate's target capabilities as specified in the project objectives.

Criteria	MagicPlan	Google Measure	Leica BLK360	RoomScan Pro	Proposed System (EchoLocate)
Primary Technology	AR + LiDAR / Manual	AR (ARCore/ARKit)	Laser LiDAR scanner	Acoustic + compass contact	Computer-vision depth (ARCore) + acoustic DSP
Floor Plan Generation	Automatic 2D/3D plan	Point-to-point only; no plan	Full 3D point cloud	Automated room outline (iOS)	Vision-based 2D plan + 3D preview
Indoor Navigation	None	None	None	None	Turn-by-turn A* navigation with AR overlay
Crowdsourced Maps	No (private cloud only)	No	No	No	Yes — community map database
Hardware Required	Smartphone (LiDAR optional)	ARCore/ARKit device	Dedicated LiDAR unit (~£15,000)	iPhone only	Any commodity Android/iOS phone
Offline / Privacy	Cloud-dependent for export	Cloud-dependent	Offline capable	Fully on-device	On-device sensing + Drift; cloud only for shared maps
Cost	Freemium (paid export)	Free	Very high (£14,000+)	Low (App Store purchase)	Free — no additional hardware
Cross-platform	iOS & Android	iOS & Android	No (proprietary)	iOS only	Android & iOS (Flutter)
Accessibility Features	None	None	None	None	Voice + haptic guidance for blind/low-vision users

2.5 DISCUSSION OF COMPARATIVE ANALYSIS

The comparative analysis reveals a clear and consistent pattern across all four reviewed systems: each tool addresses a narrow slice of the indoor spatial problem — measurement or floor-plan capture — but none provides a unified solution that maps, navigates, shares, and supports accessibility on commodity smartphones.

MagicPlan is the most feature-complete consumer tool reviewed, offering automatic 2D and 3D floor plan generation with CAD export. However, its dependence on LiDAR for high accuracy, its cloud-based architecture, and the absence of any navigation or accessibility capability disqualify it as a complete solution for users without premium devices or in privacy-sensitive contexts (Prattico et al., 2021; Sensopia, 2023).

Google Measure and its ARCore-based equivalents offer the widest device accessibility and confirm that vision-based measurement is feasible on commodity hardware — the same foundation EchoLocate adopts for depth sensing. However, they support only point-to-point measurements, cannot generate floor plans, and provide no navigation, sharing, or accessibility (Faggion et al., 2020; Google LLC, 2021).

The Leica BLK360 establishes the performance ceiling for the domain with millimetre-level laser accuracy, but at a cost exceeding £14,000 and requiring dedicated software it is entirely inaccessible to the individual users EchoLocate targets (Leica Geosystems, 2022; Gu et al.,

2015). It confirms that comprehensive indoor mapping is technically achievable while underscoring the affordability gap.

RoomScan Pro is instructive as the only reviewed system that uses smartphone audio for spatial sensing, demonstrating that acoustic methods can produce usable outputs on-device. However, its iOS exclusivity, contact-based method, lack of navigation, and complete absence of accessibility features highlight precisely the gaps EchoLocate is designed to fill (Locometric, 2022; Mirowski et al., 2013).

Taken together, the analysis confirms that no existing system simultaneously achieves affordable vision-based floor-plan generation, crowdsourced indoor maps, turn-by-turn indoor navigation, on-device privacy, and accessibility support within a single cross-platform application. EchoLocate is positioned to address all five of these dimensions.

2.6 JUSTIFICATION FOR THE PROPOSED SYSTEM

The review of related systems reveals a clear and consistent gap: no existing solution simultaneously achieves low hardware cost, cross-platform support, vision-based floor-plan generation, crowdsourced indoor maps, on-device privacy, indoor navigation, and accessibility for blind and low-vision users. MagicPlan and Google Measure depend on cloud services and provide no navigation or accessibility (Faggion et al., 2020; Prattico et al., 2021). The Leica BLK360, while highly accurate, is cost-prohibitive (Leica Geosystems, 2022; Gu et al., 2015). RoomScan Pro, the closest analogue in its use of smartphone audio, is restricted to iOS, requires physical wall contact, and offers neither navigation nor accessibility (Locometric, 2022; Mirowski et al., 2013).

EchoLocate addresses these gaps by using the ARCore Depth API to generate 2D floor plans from a commodity smartphone camera, supplemented by an acoustic DSP pipeline that classifies room type and provides a fallback estimate when vision fails — retaining a meaningful Digital Signal Processing contribution. Floor plans are shared through a crowdsourced community database, so any user can load a building mapped by others and navigate it with turn-by-turn guidance, an AR overlay, and real-time obstacle alerts. A local-first Drift architecture keeps sensing and floor-plan data on the device, and an integrated voice-and-haptic accessibility mode extends indoor navigation to blind and low-vision users — a dimension absent from all reviewed systems. This combination of affordability, crowdsourcing, navigation, privacy, accessibility, and cross-platform availability justifies the development of EchoLocate as a meaningful contribution to mobile sensing and indoor spatial computing.

2.7 RESEARCH GAPS IDENTIFIED

The systematic review of MagicPlan, Google Measure / AR Ruler Apps, the Leica BLK360, and RoomScan Pro reveals five critical research gaps that collectively leave accessible indoor spatial mapping and navigation without a satisfactory solution for commodity smartphone users. These gaps are structural absences across the existing landscape, with no single reviewed system addressing more than two simultaneously.

Gap 1: No Affordable, LiDAR-Free Vision-Based Floor Plan Generation on Commodity Phones

MagicPlan achieves high accuracy only with LiDAR on premium Apple devices, while Google Measure cannot generate floor plans at all (Prattico et al., 2021; Faggion et al., 2020). No reviewed consumer system generates complete 2D floor plans from depth-from-motion on an ordinary smartphone camera without dedicated depth hardware. This gap motivates EchoLocate's ARCore Depth API mapping engine.

Gap 2: No Crowdsourced Community Database of Indoor Maps

Every reviewed system treats a floor plan as a private artefact captured by one user for their own use; none allows users to contribute maps to a shared repository that others can reuse. Because creating indoor maps is costly, the interiors of most everyday buildings remain unmapped. No reviewed system builds an open, community-contributed database of indoor floor plans. This gap motivates EchoLocate's crowdsourced map repository.

Gap 3: No Indoor Navigation Built on User-Generated Maps

All four reviewed systems stop at measurement or floor-plan capture; none provides turn-by-turn navigation through a building, and none routes a user from one point to a destination indoors (Locometric, 2022; Sensopia, 2023). No reviewed system converts a captured floor plan into a navigable graph for indoor wayfinding. This gap motivates EchoLocate's graph-based (A*) navigation, AR waypoint overlay, and real-time obstacle detection.

Gap 4: No Accessibility-Oriented Indoor Navigation Tool

MagicPlan, Google Measure, the Leica BLK360, and RoomScan Pro are all visually oriented tools relying on screens, camera viewfinders, or physical alignment with visible surfaces (Prattico et al., 2021; Locometric, 2022). None provides haptic feedback, spatial audio cues, or voice-guided navigation for visually impaired users, despite the significant need of blind and low-vision individuals navigating unfamiliar interiors. This gap motivates EchoLocate's integrated voice-and-haptic accessibility mode.

Gap 5: No Unified, On-Device, Privacy-Preserving Multimodal System

The existing landscape is fragmented across hardware tiers, operating systems, and sensing modalities (Faggion et al., 2020; Leica Geosystems, 2022). No single system unifies vision-based floor-plan generation, acoustic room classification, crowdsourced map sharing, indoor navigation, and accessibility within one cross-platform application that performs its sensing on-device. This gap constitutes the primary academic and technical justification for the design of EchoLocate.

Table 2 below summarises the five research gaps, maps each to the reviewed system(s) that fail to address it, and identifies the EchoLocate component designed to close the gap.

Research Gap	Systems Failing to Address	EchoLocate Phase	EchoLocate Response
Gap 1: No affordable, LiDAR-free vision-based floor plans	All four (MagicPlan: LiDAR-dependent; Measure: no plans; Leica: hardware; RoomScan: iOS contact)	Phase 2	ARCore Depth API depth-from-motion mapping and line-fitting to a 2D floor plan on any commodity phone

Research Gap	Systems Failing to Address	EchoLocate Phase	EchoLocate Response
Gap 2: No crowdsourced indoor map database	All four (each treats maps as private artefacts)	Phase 4	Supabase community repository for uploading, browsing, and reusing shared floor plans
Gap 3: No navigation on user-generated maps	All four (measurement/capture only; no routing)	Phase 4	Graph-based A* routing, turn-by-turn guidance, AR overlay, and obstacle detection
Gap 4: No accessibility-oriented indoor navigation	All four (screen/contact-based; no audio/haptic guidance)	Phase 4	Voice-and-haptic accessibility mode for blind and low-vision users
Gap 5: No unified on-device multimodal system	All four (each addresses only one layer)	Phases 1–5	Single cross-platform app unifying CV mapping, acoustic DSP, crowdsourcing, navigation, and accessibility

2.8 CONCEPTUAL DESIGN

How the System Works

EchoLocate operates as a five-stage sensing, mapping, and navigation pipeline. The stages are not independent modules but interlocking layers, with each building on the data established by the one before it. All sensing and processing run on-device; the only network operation is the optional upload and download of shared maps in the crowdsourced database.

Stage 1 – Vision-Based Depth Capture

At the start of a scan, EchoLocate uses the ARCore Depth API to capture depth-from-motion data as the user walks through a space pointing the camera at the walls. ARCore fuses the camera feed with the device’s motion sensors to infer the distance to surfaces without LiDAR, accumulating a spatial point cloud of the environment. When vision-based depth is unreliable — in low light or against a featureless surface — the system falls back to a short acoustic measurement (Stage 3).

Stage 2 – Floor Plan Generation and Semantic Labelling

The accumulated point cloud is passed to the floor-plan engine, where line-fitting techniques (RANSAC and Hough transform) snap noisy points into clean wall segments to form a 2D floor plan in real time. Simultaneously, Google ML Kit runs on the camera feed to detect doors and obstacles and to read room numbers and signage, placing semantic labels on the plan. An extruded 3D preview is generated by raising the 2D walls to a default ceiling height.

Stage 3 – Acoustic Room Classification and Fallback

In parallel, EchoLocate emits a short FMCW chirp in the 16–20 kHz band and records the echo. Rather than measuring distance, the Digital Signal Processing engine applies a Fast Fourier Transform to extract reverberation features, which a TensorFlow Lite classifier uses to identify the type of space (for example corridor, small room, or large hall). The same DSP pipeline provides

a fallback distance estimate via cross-correlation when vision-based depth fails. The acoustic result enriches the map metadata and adjusts how the navigation system describes the environment.

Stage 4 – Crowdsourcing, Navigation and Accessibility

Once a plan is complete, the user can upload it to the Supabase crowdsourced database, tagged with building name and floor, where a basic quality check screens submissions. Any user can then load a building from the database, select a destination, and navigate: the floor plan is converted into a graph and an A* algorithm computes the shortest route, delivering turn-by-turn guidance with an AR waypoint overlay. During navigation, ML Kit detects obstacles in real time and announces them. For blind and low-vision users, a voice-driven accessibility mode conveys every instruction, obstacle alert, and room-type change through audio and haptic feedback.

Stage 5 – Local-First Persistence and PDF Export

All floor-plan data, session metadata, and navigation history are persisted to a local Drift database stored entirely on the device, enabling fast offline reload. When the user wishes to share a plan outside the app, a PDF generation module converts the floor plan and its metadata into a formatted report that can be shared through any standard device mechanism at the user's discretion.

Conclusion of Conceptual Design

The conceptual design of EchoLocate demonstrates a well-integrated five-stage pipeline that overcomes the limitations of the existing systems reviewed in this chapter. By unifying vision-based depth mapping, acoustic room classification, crowdsourced map sharing, graph-based indoor navigation with accessibility feedback, and local-first persistence into a single cross-platform application, the system provides a more capable, accessible, privacy-preserving, and hardware-independent solution than any comparable tool currently available. Together the five stages form a closed-loop architecture where the environment is sensed visually and acoustically, a floor plan is generated and labelled on-device, the result is shared through the community database, and any user can navigate the space — by sight or by sound — realising EchoLocate's objective of democratising indoor mapping and navigation for any smartphone user.

2.9 SUMMARY

This chapter presented a structured review of four existing systems relevant to the EchoLocate project: MagicPlan, Google Measure / AR Ruler Apps, the Leica BLK360, and RoomScan Pro. For each system, key features, good attributes, and notable limitations were identified and discussed. A comparative analysis table evaluated all four systems against EchoLocate's target capabilities across nine functional criteria, followed by a discussion of the findings.

The review revealed that while current systems offer useful but limited functionality, they are constrained by hardware dependency, platform exclusivity, cloud reliance, the absence of indoor navigation, the absence of crowdsourced maps, and poor accessibility. The identification of five research gaps formally established the technical and academic justification for EchoLocate's design. The proposed system addresses all five gaps by combining vision-based floor-plan generation with acoustic room classification, crowdsourced map sharing, graph-based navigation,

and an accessibility mode, delivered as a cross-platform, privacy-preserving application. The conceptual design in Section 2.8 described the five-stage pipeline through which these goals are realised, providing a foundation for the system architecture and implementation decisions detailed in the chapters that follow.

REFERENCES

- Faggion, F., Dominici, D., Grussenmeyer, P., & Tournadre, V. (2020). Handheld devices and apps for 3D indoor mapping: An overview. *Remote Sensing*, 12(10), 1621. <https://doi.org/10.3390/rs12101621>
- Google LLC. (2021). Google Measure AR app – ARCore developer documentation. Google Developers. <https://developers.google.com/ar/develop/>
- Gu, Y., Lo, A., & Niemegeers, I. (2015). A survey of indoor positioning systems for wireless personal networks. *IEEE Communications Surveys & Tutorials*, 11(1), 13–32. <https://doi.org/10.1109/SURV.2009.090103>
- Leica Geosystems. (2022). Leica BLK360 imaging laser scanner – Technical specifications. Hexagon AB. <https://leica-geosystems.com/products/laser-scanners/scanners/blk360>
- Locometric. (2022). RoomScan Pro – Floor plan app for iPhone and iPad. Apple App Store. <https://apps.apple.com/app/roomscan-pro/id673673795>
- Mirowski, P., Ho, T. K., Yi, S., & MacDonald, M. (2013). SignalSLAM: Simultaneous localisation and mapping with mixed WiFi, Bluetooth, LTE and magnetic signals. In *Proceedings of the 2013 International Conference on Indoor Positioning and Indoor Navigation (IPIN)* (pp. 1–8). IEEE. <https://doi.org/10.1109/IPIN.2013.6817853>
- Prattico, F. G., Lamberti, F., Cannavaro, I., Morra, L., & Montuschi, P. (2021). Comparing state-of-the-art and emerging augmented reality interfaces for autonomous vehicle external human–machine interaction. *IEEE Transactions on Vehicular Technology*, 70(3), 2487–2500. <https://doi.org/10.1109/TVT.2021.3054338>
- Sensopia. (2023). MagicPlan – Floor plan & home design application. App Store & Google Play. <https://www.sensopia.com/>

CHAPTER THREE: METHODOLOGY

3.1 Introduction

This chapter presents the methodology used to design and develop EchoLocate, a cross-platform mobile application that generates 2D indoor floor plans from a commodity smartphone and provides turn-by-turn indoor navigation for both sighted and visually impaired users. It covers the system architecture, component design, requirements, software process model, logical design, and development tools, all shaped by the constraints identified in Chapter One: operation on commodity phones without LiDAR or dedicated depth hardware, on-device privacy-preserving sensing, and accessibility for blind and low-vision users. The design directly responds to the five research gaps established in Chapter Two, namely the absence of affordable LiDAR-free floor-plan generation, the absence of a crowdsourced indoor-map database, the lack of navigation built on user-generated maps, the lack of accessibility-oriented indoor navigation, and the absence of a unified on-device multimodal system.

3.2 Architecture of the Proposed System

EchoLocate adopts a local-first, layered client architecture with a thin cloud tier, allowing each layer to be developed and tested independently. The presentation layer is a Flutter application (Riverpod state management, Go Router navigation) that renders the scan, explore, 2D navigation, and 3D preview screens and drives the voice-and-haptic accessibility mode. The on-device sensing and processing layer is the analytical core of the system: it accumulates depth-from-motion data from the ARCore Depth API into a spatial point cloud, fits wall segments using a RANSAC and Hough-transform pipeline, runs Google ML Kit object and text detection to label doors, obstacles, and signs, and executes a Dart-native FMCW acoustic pipeline whose FFT-based reverberation features classify the room type and provide a fallback distance estimate when vision fails. The persistence layer uses a local Drift (SQLite) database so that floor plans, scan sessions, and navigation history remain fully available offline. A thin cloud tier built on Supabase backs the crowdsourced map repository, allowing users to upload the floor plans they scan and to browse, load, and navigate buildings mapped by others. All heavy sensing and floor-plan computation runs on the device; the cloud is used only for sharing finished maps, keeping the system privacy-preserving by design.

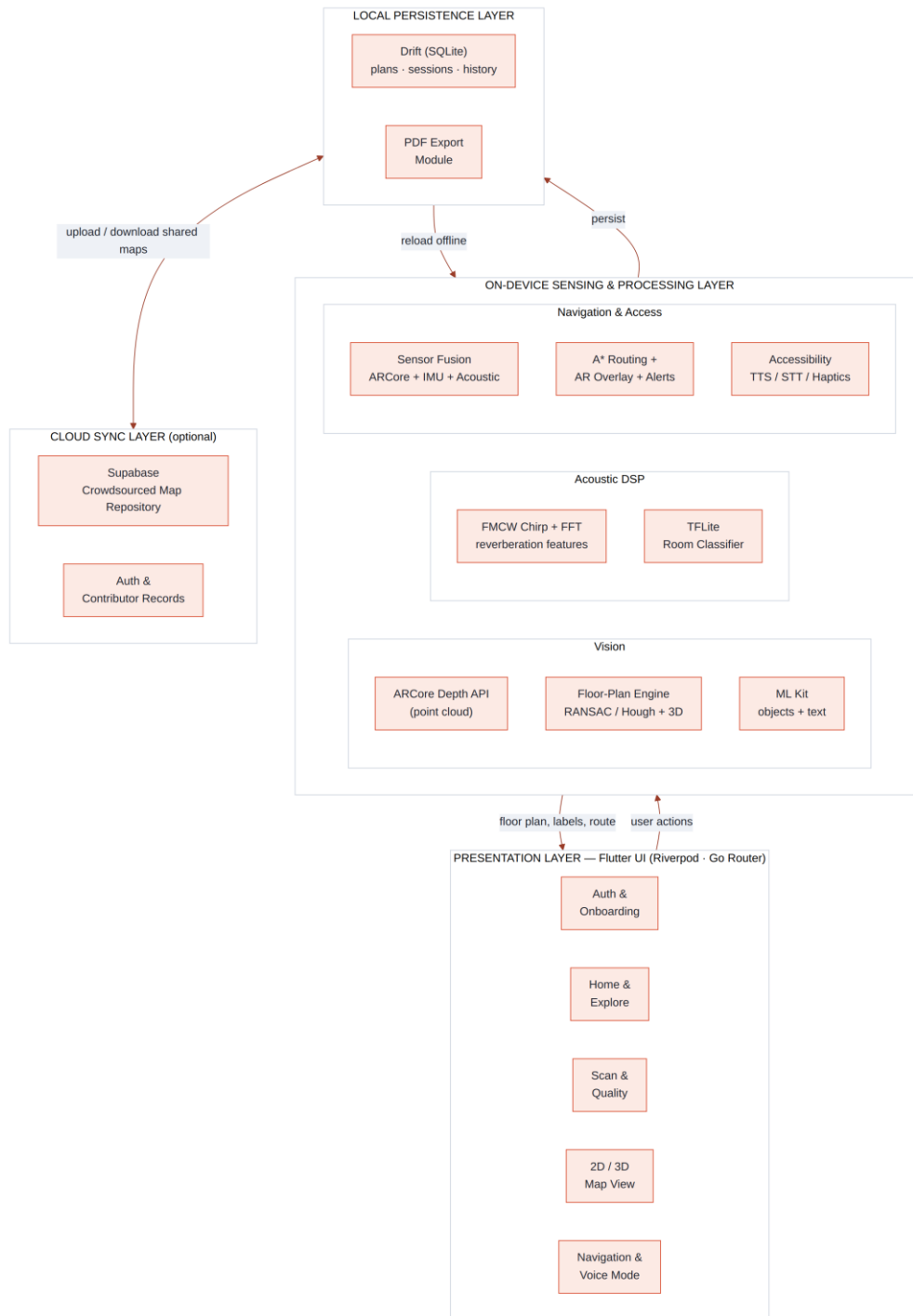


Figure 3.1: EchoLocate Layered System Architecture (On-Device Sensing with Crowdsourced Cloud Tier)

3.3 Component Design and Descriptions

EchoLocate comprises six core components with clearly defined responsibilities, each mapping to one or more of the specific objectives defined in Chapter One.

Component	Function
Vision Depth-Mapping Engine	Captures depth-from-motion frames through the ARCore Depth API via a native Android platform channel and accumulates them into a spatial point cloud. Provides the raw geometry from which wall segments are derived, with no dependency on LiDAR or a dedicated depth sensor.
Floor-Plan Generator	Converts the accumulated point cloud into clean 2D wall segments using a RANSAC and Hough-transform line-fitting pipeline, and produces an extruded 3D preview of the captured space for visual confirmation.
Semantic Detection Module	Runs Google ML Kit object detection and text recognition on the live camera feed to identify and label doors, obstacles, and room or sign text, placing these features onto the generated floor plan entirely on-device.
Acoustic DSP Pipeline	Emits an FMCW chirp and applies a Dart-native Fast Fourier Transform to the captured echo, extracting reverberation features that a TensorFlow Lite classifier maps to a room type (corridor, small room, or large hall). Supplies a fallback distance estimate when vision-based depth fails in low light or against featureless surfaces, retaining the project's Digital Signal Processing contribution.
Navigation Engine	Converts a loaded floor plan into a navigable graph and computes the shortest path using a custom Dart A* routing engine, driving turn-by-turn guidance, an AR waypoint overlay, and real-time obstacle alerts.
Crowdsource & Persistence Service	Manages local-first storage through Drift and synchronises shared maps with the Supabase repository, handling upload, browsing, loading, map-quality indicators, and PDF export of floor-plan reports.

Table 3.1: EchoLocate System Components

The acoustic pipeline is parameterised for reliable on-device operation across the smartphone speakers and microphones available on commodity hardware. Table 3.2 summarises the key FMCW signal parameters used by the Acoustic DSP Pipeline.

Parameter	Value / Range	Purpose
Chirp type	Linear FMCW (up-sweep)	Provides a wide-band excitation whose frequency-to-time mapping yields range from the beat frequency.
Frequency sweep	2 kHz – 8 kHz	Kept within the reliable response band of commodity phone speakers and microphones while remaining largely inaudible-adjacent and robust to noise.
Chirp duration	40 ms per sweep	Long enough for adequate frequency resolution, short enough to limit room-mode smearing.
Sampling rate	44.1 kHz	Standard mobile audio rate; satisfies the Nyquist criterion for the 8 kHz upper bound.
FFT window	2048-point, Hann-windowed	Balances spectral resolution against latency for real-time reverberation feature extraction.
Feature set	RT60 estimate, spectral decay, energy-decay slope	Reverberation descriptors fed to the TFLite classifier to predict room type and inform the fallback distance estimate.

Table 3.2: FMCW Acoustic Sensing Parameters

The Semantic Detection Module targets a small, navigation-relevant set of object and text classes. Table 3.3 lists the detection targets and how each is used on the floor plan.

Detection Target	ML Kit Capability	Use in EchoLocate
Doors and doorways	Object detection	Marked as graph connection points between spaces so the navigation engine can route between rooms.
Obstacles (furniture, pillars)	Object detection	Flagged on the floor plan and used for real-time obstacle alerts during navigation.
Room numbers and signs	Text recognition (OCR)	Attached as semantic labels to spaces, enabling destination search by room name or number.
Directional / exit signage	Text recognition (OCR)	Used to annotate exits and assist wayfinding, including spoken cues in accessibility mode.

Table 3.3: On-Device Semantic Detection Targets (Google ML Kit)

3.4 Evaluation and Analysis of Software System Type

EchoLocate is a mobile-first, sensor-driven spatial computing application. It functions primarily as a stand-alone, on-device application: floor-plan generation, acoustic classification, semantic detection, navigation, and persistence all execute locally and remain fully available offline. It also has an interactive, transaction-based dimension when online, where it behaves as a client to the Supabase crowdsourced repository for uploading and retrieving shared maps. Because it is a real-time, sensor-fusion and assistive-navigation tool rather than a regulated safety system, the design treats navigation output as guidance that complements the user's own judgement, provides redundant feedback through visual, audio, and haptic channels, and degrades gracefully to acoustic fallback sensing when vision is unavailable.

3.5 Functional Requirements of the Proposed System

The functional requirements below are derived from the specific objectives in Chapter One and each maps to at least one of the five research gaps identified in Chapter Two.

- FR-01: Capture depth-from-motion data through the ARCore Depth API on commodity Android smartphones and accumulate it into a spatial point cloud, without LiDAR or a dedicated depth sensor.
- FR-02: Generate a clean 2D floor plan from the accumulated point cloud using a RANSAC and Hough-transform line-fitting pipeline, and render an extruded 3D preview of the space.
- FR-03: Detect and label doors, obstacles, and room or sign text on the live camera feed using on-device Google ML Kit, placing them on the generated floor plan.
- FR-04: Emit an FMCW chirp and apply an FFT pipeline to classify room type and provide a fallback distance estimate when vision-based depth sensing fails.
- FR-05: Allow users to upload scanned floor plans to a crowdsourced repository and to browse, load, and navigate maps contributed by other users.
- FR-06: Compute turn-by-turn indoor routes over a loaded floor plan using a graph-based A* engine, with an AR waypoint overlay and real-time obstacle alerts.
- FR-07: Provide a fully voice-driven accessibility mode that conveys navigation guidance through spoken instructions and haptic feedback for blind and low-vision users.

- FR-08: Persist floor plans, scan sessions, and navigation history locally via the Drift database for fully offline operation.
- FR-09: Export a generated floor plan as a shareable PDF report.

3.6 Identification of Stakeholders

Five stakeholder groups are identified for EchoLocate. Sighted end users are the primary users who scan spaces, generate floor plans, and follow turn-by-turn navigation with the AR overlay; their needs drive the core scanning and routing workflows. Blind and low-vision users are an equally central group whose requirements shape the voice-and-haptic accessibility mode, the redundant audio cues, and the obstacle-alert design — a dimension absent from all systems reviewed in Chapter Two. Community contributors are the users who upload scanned maps to the crowdsourced repository; the value of the shared database depends directly on their participation, so the system must make contribution and map-quality feedback simple and rewarding. Building administrators (for example facility or campus managers) are indirect stakeholders interested in having accurate, navigable maps of their premises and in the accessibility benefits the system brings to visitors. The developer and maintainer is the final stakeholder, responsible for maintaining the Supabase backend, updating the acoustic classifier model, and resolving deployment issues after handover.

3.7 Requirement Gathering Process

Requirements were gathered through three complementary approaches. Literature and gap synthesis drew on the systematic review in Chapter Two, where MagicPlan, Google Measure, the Leica BLK360, and RoomScan Pro were evaluated and five research gaps were established; each gap was mapped to at least one functional requirement in Section 3.5, ensuring the design responds to documented absences rather than assumed needs. Comparative system analysis examined these four existing tools against the capabilities EchoLocate targets — affordability, cross-platform support, floor-plan generation, crowdsourcing, navigation, privacy, and accessibility — to identify functionality worth replicating and limitations worth addressing. Standards and guidelines review consulted the Web Content Accessibility Guidelines (WCAG 2.1) to shape the accessibility mode and the redundant non-visual feedback requirements, and Ghana’s Data Protection Act 2012 (Act 843) to justify the on-device, data-minimising sensing design.

3.8 Requirement Specification

3.8.1 Functional User Requirements

ID	User Requirement	Stakeholder
FUR-01	As a user, I want to scan a space with my phone camera so that a 2D floor plan of the area is generated automatically.	Sighted User
FUR-02	As a user, I want the app to recognise doors, obstacles, and room signs so that the floor plan is labelled and searchable.	Sighted User
FUR-03	As a user, I want to upload the maps I scan and browse maps shared by others so that I can navigate buildings I have never mapped myself.	Community Contributor
FUR-04	As a user, I want turn-by-turn directions to a room within a loaded building so that I can find my destination indoors.	Sighted User
FUR-05	As a blind or low-vision user, I want spoken directions and haptic feedback so that I can navigate indoors without looking at the screen.	Blind / Low-Vision User

ID	User Requirement	Stakeholder
FUR-06	As a user, I want the app to keep working without internet so that I can scan and navigate in areas with poor connectivity.	Sighted / Low-Vision User
FUR-07	As a user, I want to export a floor plan as a PDF so that I can share or archive it.	Sighted User

Table 3.4: Functional User Requirements

3.8.2 Functional System Requirements 3

ID	System Requirement	Satisfies
FSR-01	The system shall capture ARCore depth frames and accumulate them into a point cloud, rendering a live coverage indicator during scanning.	FUR-01
FSR-02	The system shall fit 2D wall segments from the point cloud using RANSAC and Hough-transform line fitting and produce an extruded 3D preview.	FUR-01
FSR-03	The system shall run on-device ML Kit object and text detection and attach door, obstacle, and label features to the floor plan.	FUR-02
FSR-04	The system shall classify room type from FMCW reverberation features via a TFLite model and provide an acoustic fallback distance estimate when vision fails.	FUR-01, FUR-06
FSR-05	The system shall upload floor plans to the Supabase repository and list, load, and open community maps with a map-quality indicator.	FUR-03
FSR-06	The system shall build a navigation graph from a loaded plan and compute the shortest route with A*, presenting turn-by-turn steps and an AR overlay.	FUR-04
FSR-07	The system shall deliver spoken guidance through flutter_tts and haptic cues, and accept voice commands via speech_to_text in accessibility mode.	FUR-05
FSR-08	The system shall persist plans, sessions, and history in a local Drift database and export a floor plan as a PDF report.	FUR-06, FUR-07

Table 3.5: Functional System Requirements

3.8.3 UML Diagrams

Use Case Diagram 1 — Mobile Application (Front-End)

The front-end use case diagram captures the interactions available to the Sighted User and the Blind / Low-Vision User with the EchoLocate mobile application, including scanning a space, generating and previewing a floor plan, navigating, and using the accessibility voice mode.

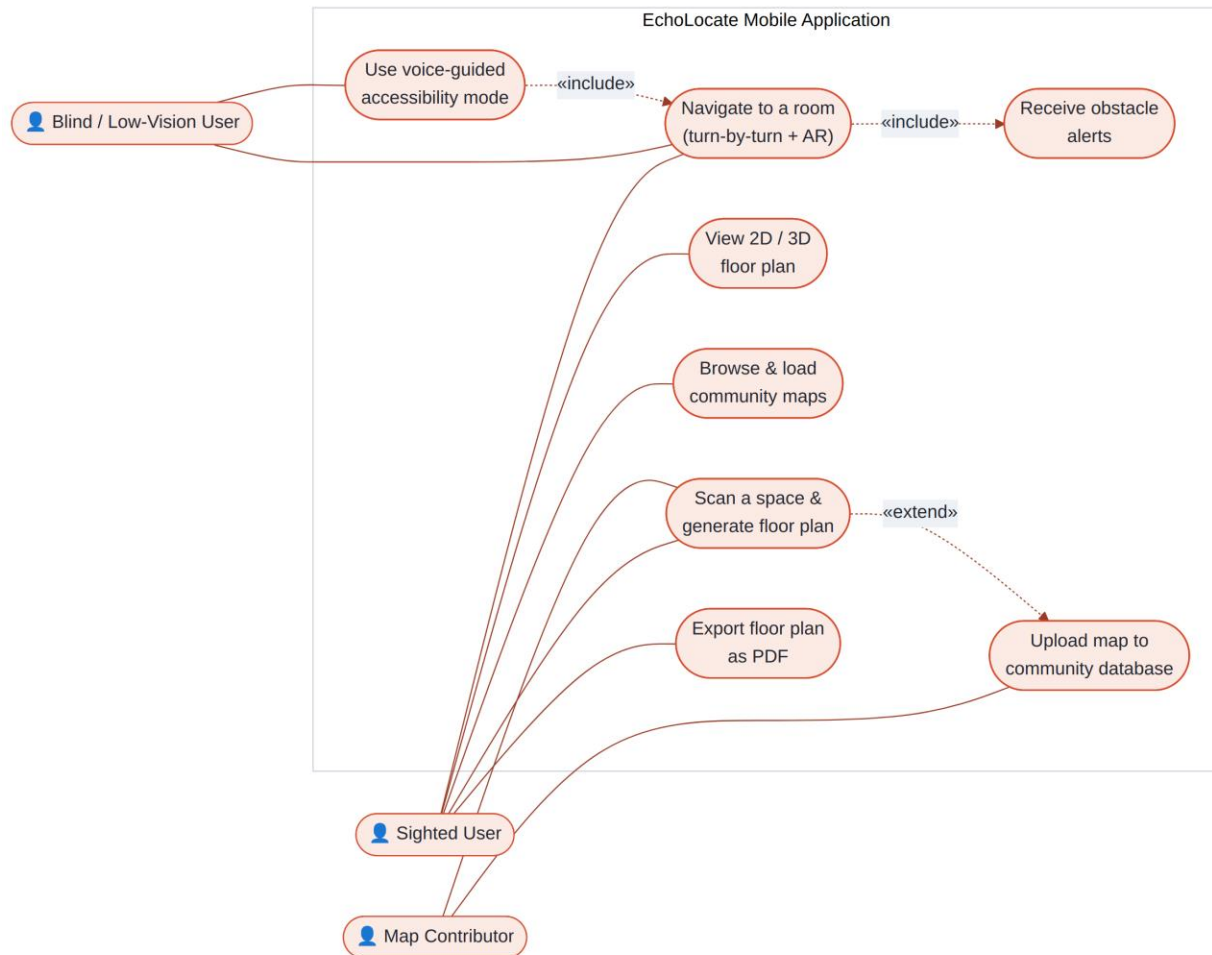


Figure 3.2: Use Case Diagram — EchoLocate Mobile Application (Front-End)

Use Case Diagram 2 — Crowdsourcing Backend (Back-End)

The back-end use case diagram captures the interactions involving the Supabase crowdsourced repository, namely uploading a scanned map, browsing and loading community maps, retrieving map-quality ratings, and synchronising contributor records.

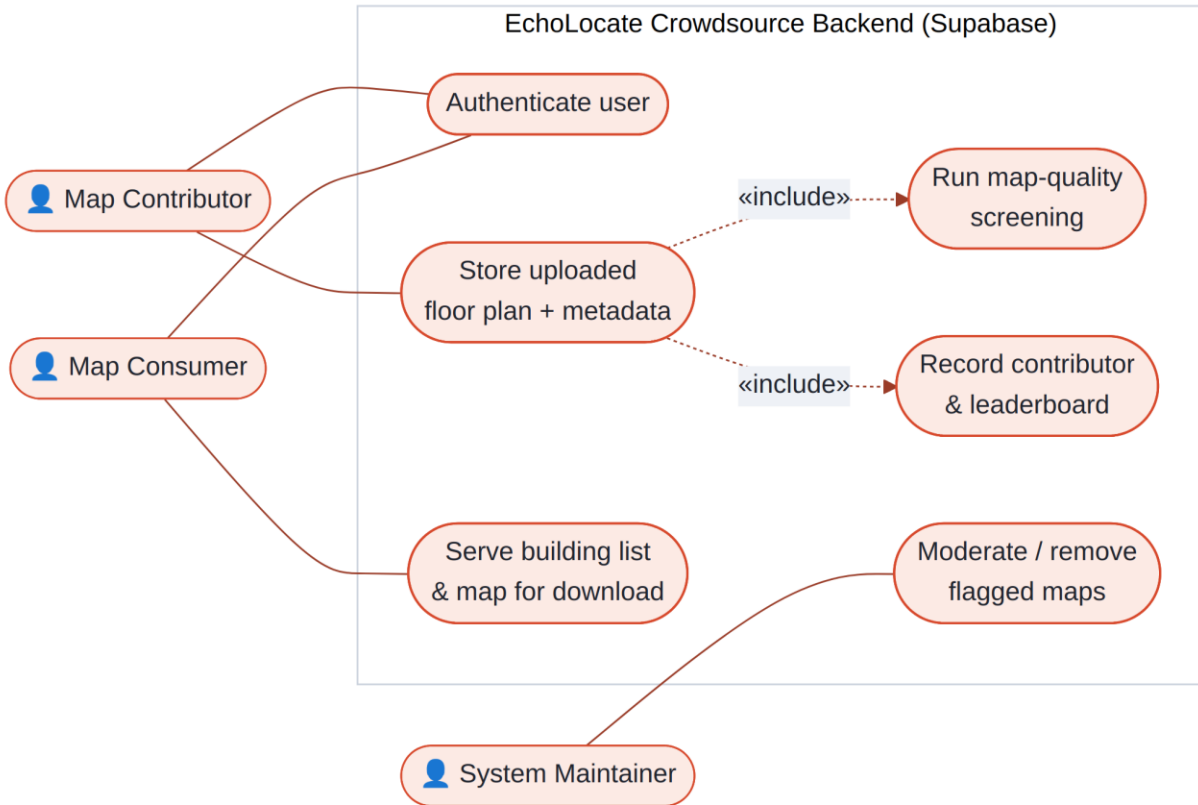


Figure 3.3: Use Case Diagram — EchoLocate Crowdsource Backend (Back-End)

Use Case Description — UC-01: Scan and Generate Floor Plan

Element	Description
Actor(s)	Sighted User, Mobile Device Sensors (ARCore, Camera, Microphone)
Precondition	Camera and motion permissions are granted and the device supports the ARCore Depth API.
Main Flow	1. User starts a scan and slowly moves the phone across the space. 2. System accumulates depth-from-motion points into a point cloud and shows live coverage quality. 3. ML Kit detects and labels doors, obstacles, and signs from the camera feed. 4. System fits 2D wall segments via RANSAC/Hough and renders the floor plan with an extruded 3D preview. 5. User saves the plan to the local Drift database.
Alternative Flow	If vision-based depth degrades in low light or against featureless surfaces, the system invokes the acoustic FMCW fallback to estimate distance and classify room type, and prompts the user to rescan affected areas.
Postcondition	A labelled 2D floor plan and 3D preview are generated and persisted locally, ready for navigation or upload.
Performance	A floor plan for a single room shall be generated within a few seconds of completing the scan sweep on a mid-range device.

Table 3.6: Use Case Description — UC-01: Scan and Generate Floor Plan

Use Case Description — UC-02: Navigate to a Room

Element	Description
Actor(s)	Sighted User, Blind / Low-Vision User
Precondition	A floor plan for the building has been loaded, either scanned locally or downloaded from the crowdsourced repository.
Main Flow	1. User selects or speaks a destination room. 2. System builds a navigation graph from the floor plan and computes the shortest route using A*. 3. System presents turn-by-turn steps with a 2D map and an AR waypoint overlay. 4. ML Kit raises real-time obstacle alerts as the user proceeds. 5. System announces arrival at the destination.
Alternative Flow	In accessibility mode the route is delivered entirely through spoken instructions and haptic cues (for example “Turn right in 5 metres”), the screen overlay is optional, and voice commands control the session.
Postcondition	The user is guided from the start point to the destination, with the navigation session recorded in local history.
Performance	Route computation shall complete within one second for a typical single-building floor plan after the destination is selected.

Table 3.7: Use Case Description — UC-02: Navigate to a Room

Activity Diagram — Scan and Floor-Plan Generation

The activity diagram below models the control flow of the scanning workflow, from depth capture and semantic detection through line fitting to the acoustic fallback branch and final persistence of the floor plan.

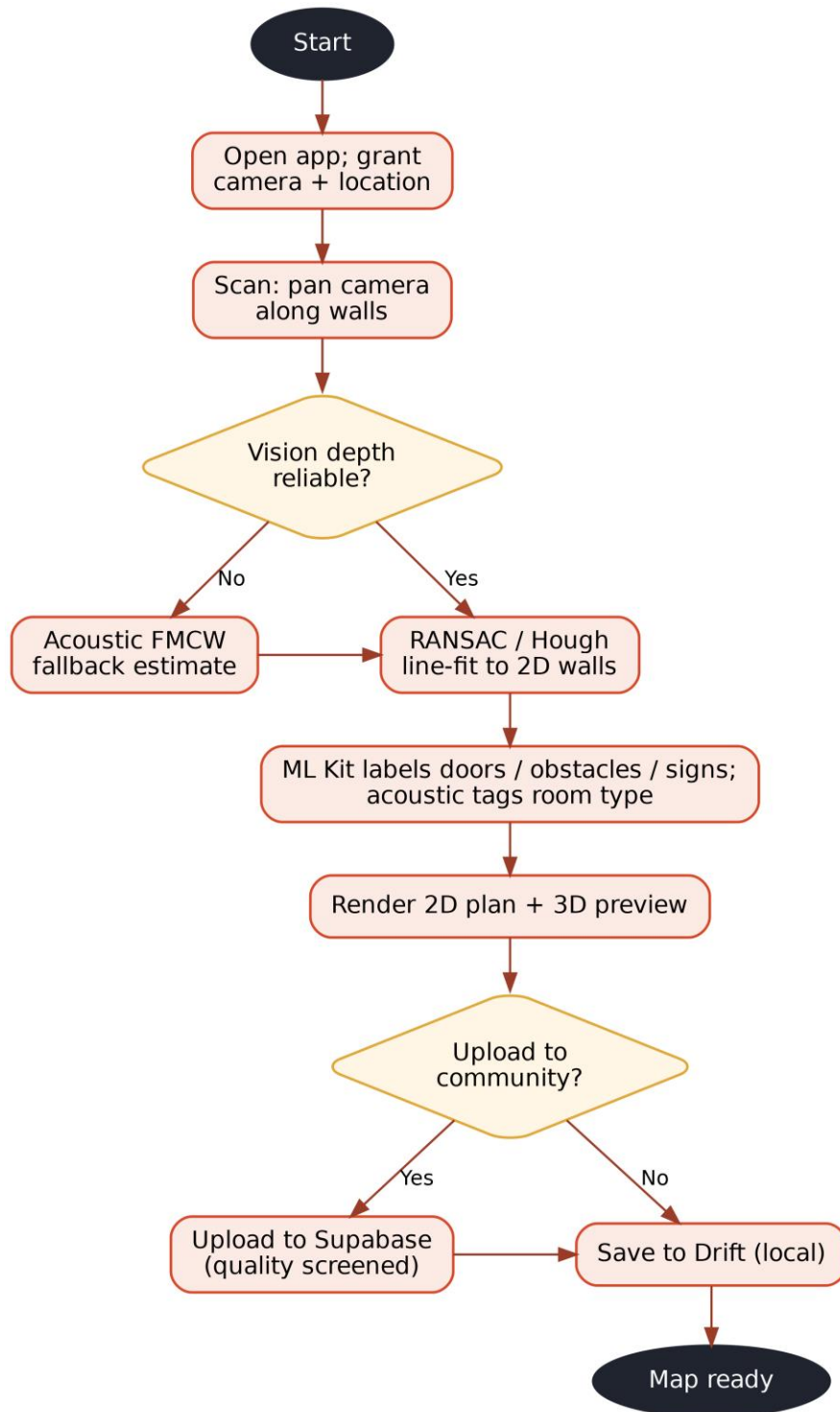


Figure 3.4: Activity Diagram — Scan and Floor-Plan Generation Flow

Activity Diagram — Indoor Navigation

The second activity diagram models the navigation workflow, from loading a map and selecting a destination through A* routing, turn-by-turn guidance, obstacle alerts, and the accessibility voice-and-haptic branch.

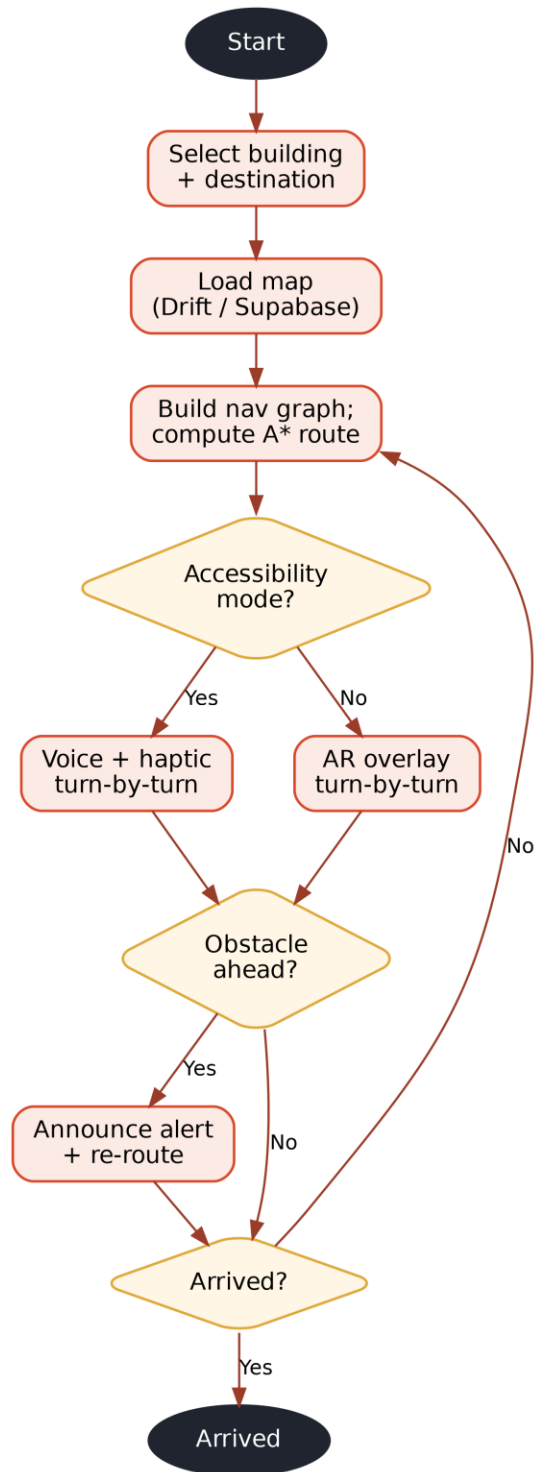


Figure 3.5: Activity Diagram — Indoor Navigation Flow

Sequence Diagram — Scan and Generate Floor Plan

The sequence diagram for the scanning workflow shows the message exchange between the user, the Flutter UI, the Vision Depth-Mapping Engine, the Semantic Detection Module, the Floor-Plan Generator, the Acoustic DSP Pipeline, and the local Drift store.

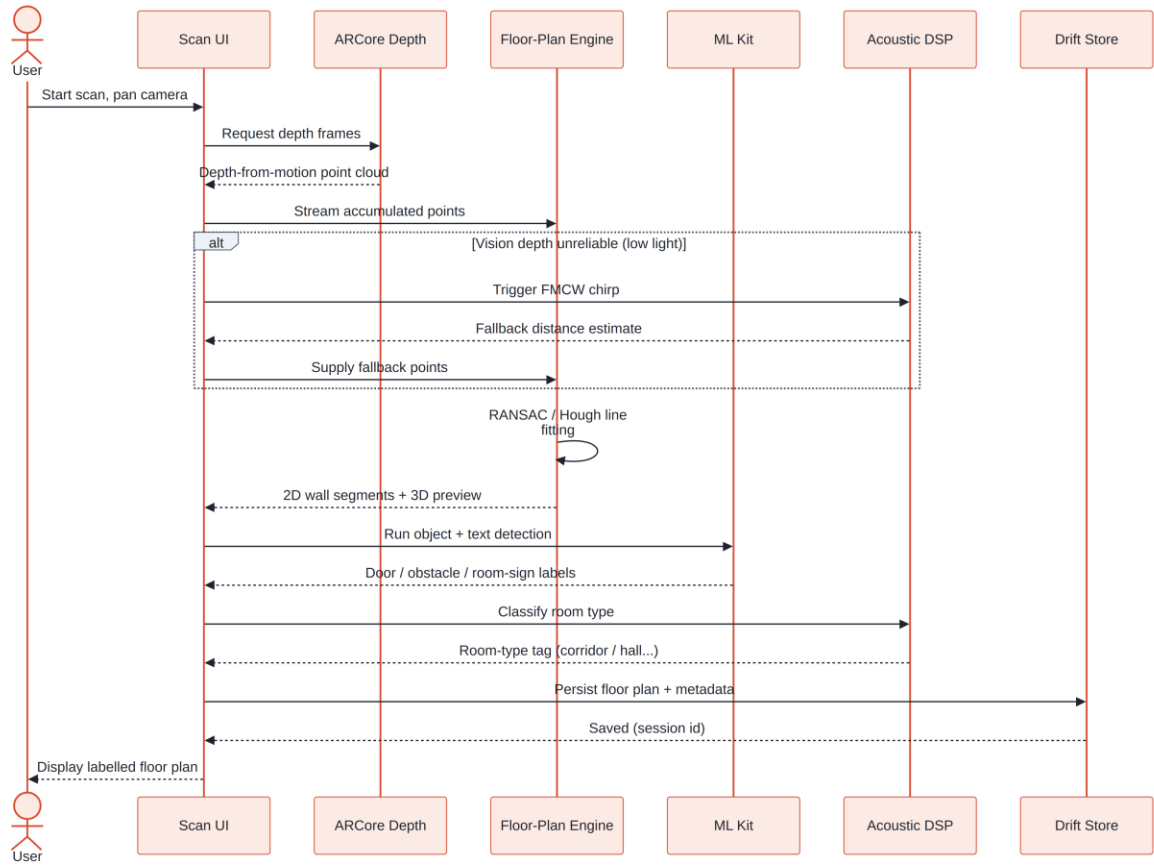


Figure 3.6: Sequence Diagram — Scan and Generate Floor Plan

Sequence Diagram — Navigate to a Room

The sequence diagram for navigation shows the interaction between the user, the Flutter UI, the Map Repository, the Navigation Engine, the Accessibility Service, and the Semantic Detection Module during a routing session.

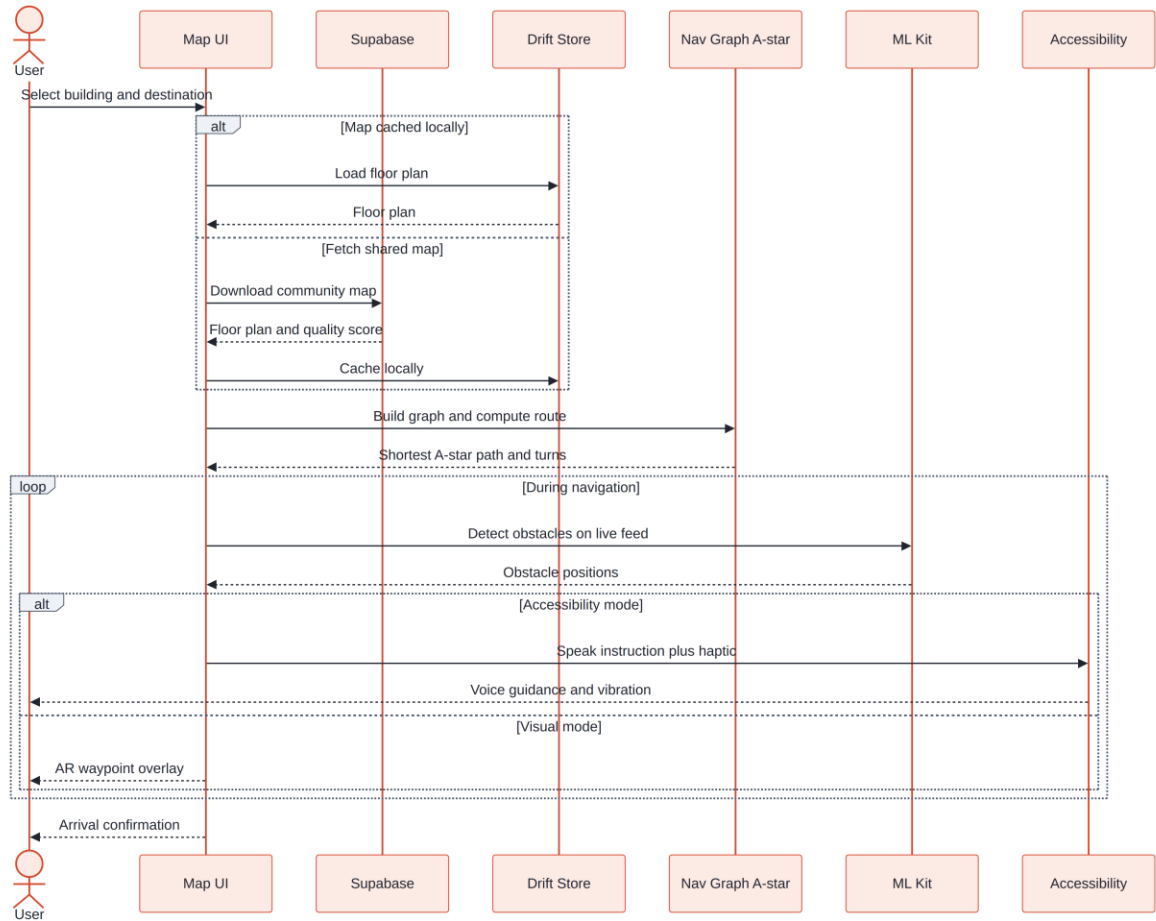


Figure 3.7: Sequence Diagram — Navigate to a Room

3.9 Non-Functional Requirements

ID	Category	Requirement	Justification
NFR-01	Performance	Route computation shall complete within 1 second for a typical single-building floor plan on a mid-range Android device.	Consistent with user tolerance thresholds (Nielsen, 1994); responsiveness is essential for live wayfinding.
NFR-02	Performance	Floor-plan generation for a single room shall complete within a few seconds of the scan sweep ending.	Keeps the scan-and-confirm loop tight enough for practical field use on commodity hardware.
NFR-03	Reliability	Scanning, floor-plan generation, navigation, and persistence shall remain fully functional with no internet connectivity.	Many indoor environments have weak or no connectivity; offline operation is a core, not optional, requirement.
NFR-04	Reliability	When vision-based depth fails, the system shall switch to the acoustic fallback without aborting the scan.	Graceful degradation ensures low light or featureless surfaces do not end the session.
NFR-05	Usability / Accessibility	All navigation guidance shall be available through redundant visual, audio, and haptic channels.	Satisfies WCAG 2.1 (W3C, 2018) and makes the system usable by blind and low-vision users.

ID	Category	Requirement	Justification
NFR-06	Portability	The application shall run from a single Flutter codebase on Android (primary) and iOS.	Cross-platform reach is a stated objective and addresses the platform-exclusivity gap from Chapter Two.
NFR-07	Privacy / Security	All sensing and floor-plan computation shall occur on-device; only finished maps the user chooses to share are uploaded.	Complies with the data-minimisation principle of Ghana's Data Protection Act 2012 (Act 843).

Table 3.8: Non-Functional Requirements with Justifications

3.10 Candidate Classes and UML Class Diagram

The following candidate classes are identified from the system's functional requirements. Each encapsulates a distinct responsibility with well-defined interfaces to adjacent classes.

Class	Layer	Responsibility
ScanSession	Sensing	Represents one scanning run; aggregates the captured point cloud, acoustic samples, and detected semantic labels.
PointCloud	Sensing	Stores accumulated depth-from-motion points and provides access for line fitting.
FloorPlan	Mapping	Central data entity; holds the set of wall segments, semantic labels, and metadata for a mapped space.
WallSegment	Mapping	A fitted 2D line segment produced by the RANSAC/Hough pipeline; the building block of a FloorPlan.
SemanticLabel	Mapping	A detected door, obstacle, or text label with position, type, and recognised text.
AcousticSample	Sensing	An FMCW capture with extracted reverberation features used for room-type classification and fallback distance.
RoomClassifier	DSP	Wraps the TFLite model; maps reverberation features to a room-type prediction. Provides <code>classify()</code> → <code>RoomType</code> .
NavGraph	Navigation	Graph representation derived from a FloorPlan, with nodes and weighted edges for routing.
Route	Navigation	Value object: ordered list of waypoints and turn-by-turn instructions produced by A*.
NavigationController	Navigation	Drives an active navigation session, advancing steps and requesting obstacle checks.
AccessibilityService	Presentation	Converts route steps into spoken guidance and haptic cues and handles voice commands.
MapRepository	Persistence	Local-first CRUD over the Drift database for FloorPlan and ScanSession objects.
CrowdsourcingService	Cloud	Uploads and retrieves shared FloorPlans and map-quality ratings via Supabase.
PdfExporter	Persistence	Renders a FloorPlan into a shareable PDF report.

Table 3.9: EchoLocate Candidate Classes

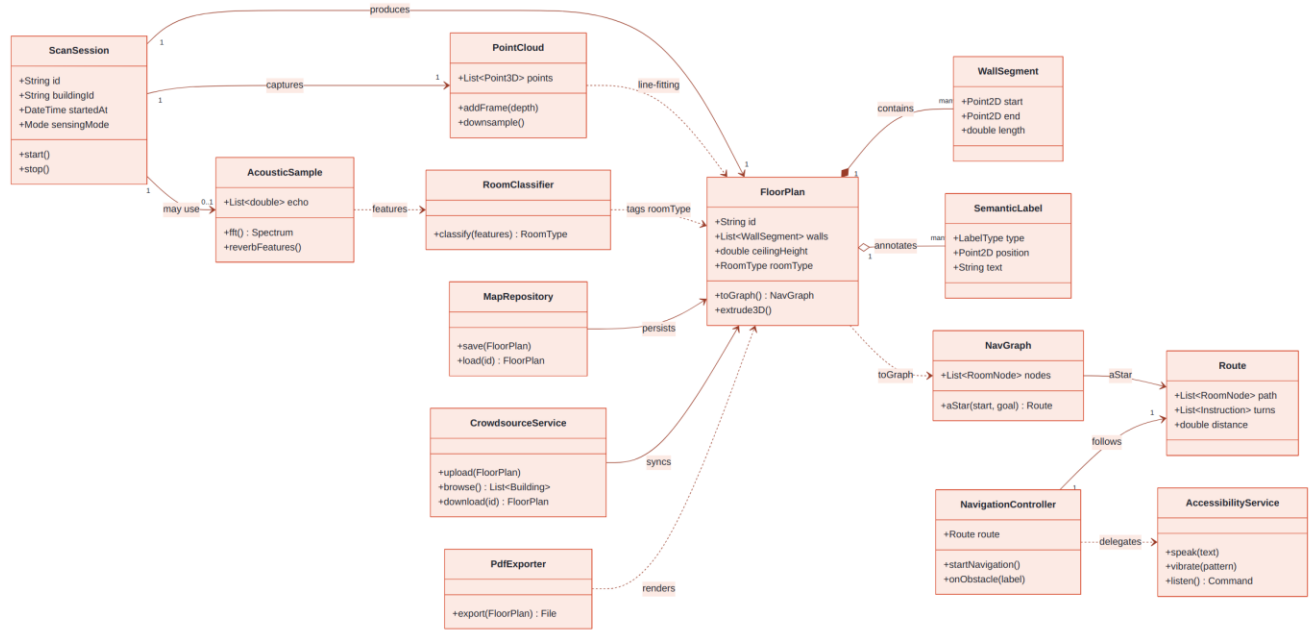


Figure 3.8: UML Class Diagram — EchoLocate System

Key relationships: FloorPlan is the central entity, composed of many WallSegment and SemanticLabel objects and produced by a ScanSession that owns a PointCloud and one or more AcousticSample objects. RoomClassifier consumes AcousticSample features to label spaces. NavGraph is built from a FloorPlan, and NavigationController uses a Route computed over the NavGraph while delegating non-visual output to AccessibilityService. MapRepository persists FloorPlan and ScanSession objects locally, while CrowdsourcingService synchronises selected FloorPlans with the cloud, and PdfExporter renders a FloorPlan on demand.

3.11 Algorithms for the System

3.11.1 Algorithm 1: Floor-Plan Generation

Step	Operation
Input	Accumulated point cloud P from the ARCore Depth API; set of ML Kit semantic detections S.
Step 1	Project the 3D points in P onto the horizontal plane to obtain a 2D point set P'.
Step 2	Cluster P' and iteratively fit dominant straight lines using RANSAC to reject outliers and noisy points.
Step 3	Apply the Hough transform to consolidate collinear inliers into candidate wall lines and estimate their orientation.
Step 4	Snap near-parallel and near-perpendicular lines to a consistent grid, trim and join endpoints to form closed WallSegment chains.
Step 5	Attach each SemanticLabel from S (doors, obstacles, signs) to its nearest wall segment or floor position.
Step 6	Extrude the 2D segments to build the 3D preview mesh.
Output	A FloorPlan object comprising wall segments, semantic labels, and a 3D preview.

Table 3.10: Algorithm 1 — Floor-Plan Generation

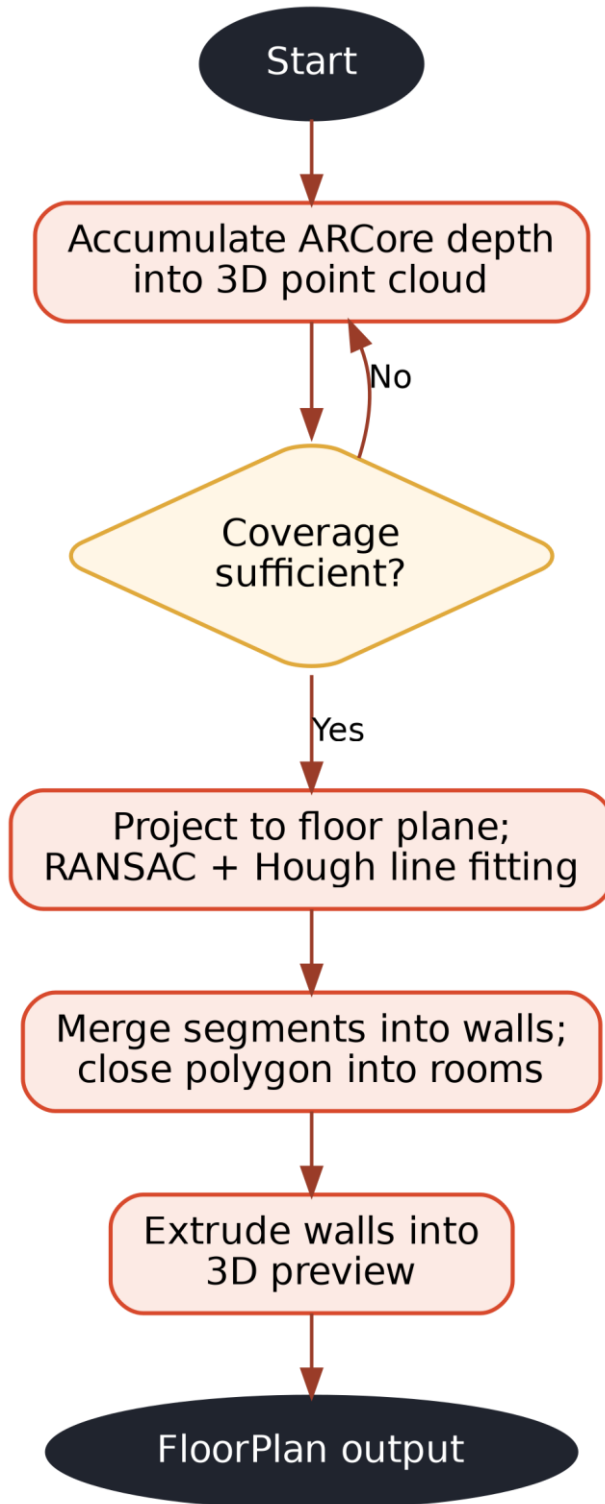


Figure 3.9: Flowchart — Floor-Plan Generation Algorithm

3.11.2 Algorithm 2: Acoustic Room Classification and Fallback

Step	Operation
Input	Vision-depth confidence v ; microphone access; trigger condition (room labelling or vision failure).
Step 1	Emit an FMCW chirp (2–8 kHz sweep, 40 ms) through the device speaker and record the echo.
Step 2	Apply a Hann-windowed 2048-point FFT to the recorded signal and extract reverberation features (RT60 estimate, spectral decay, energy-decay slope).
Step 3	Pass the feature vector to the TFLite RoomClassifier $\rightarrow$ room-type probabilities; SET $\text{roomType} = \text{argmax}$.
Step 4	IF $v < \text{threshold}$ (vision failed): estimate distance to the nearest surface from the beat frequency and return it as a fallback measurement.
Step 5	IF $v \geq \text{threshold}$: return only the room-type label to annotate the FloorPlan.
Output	A room-type classification and, when vision has failed, a fallback distance estimate.

Table 3.11: Algorithm 2 — Acoustic Room Classification and Fallback

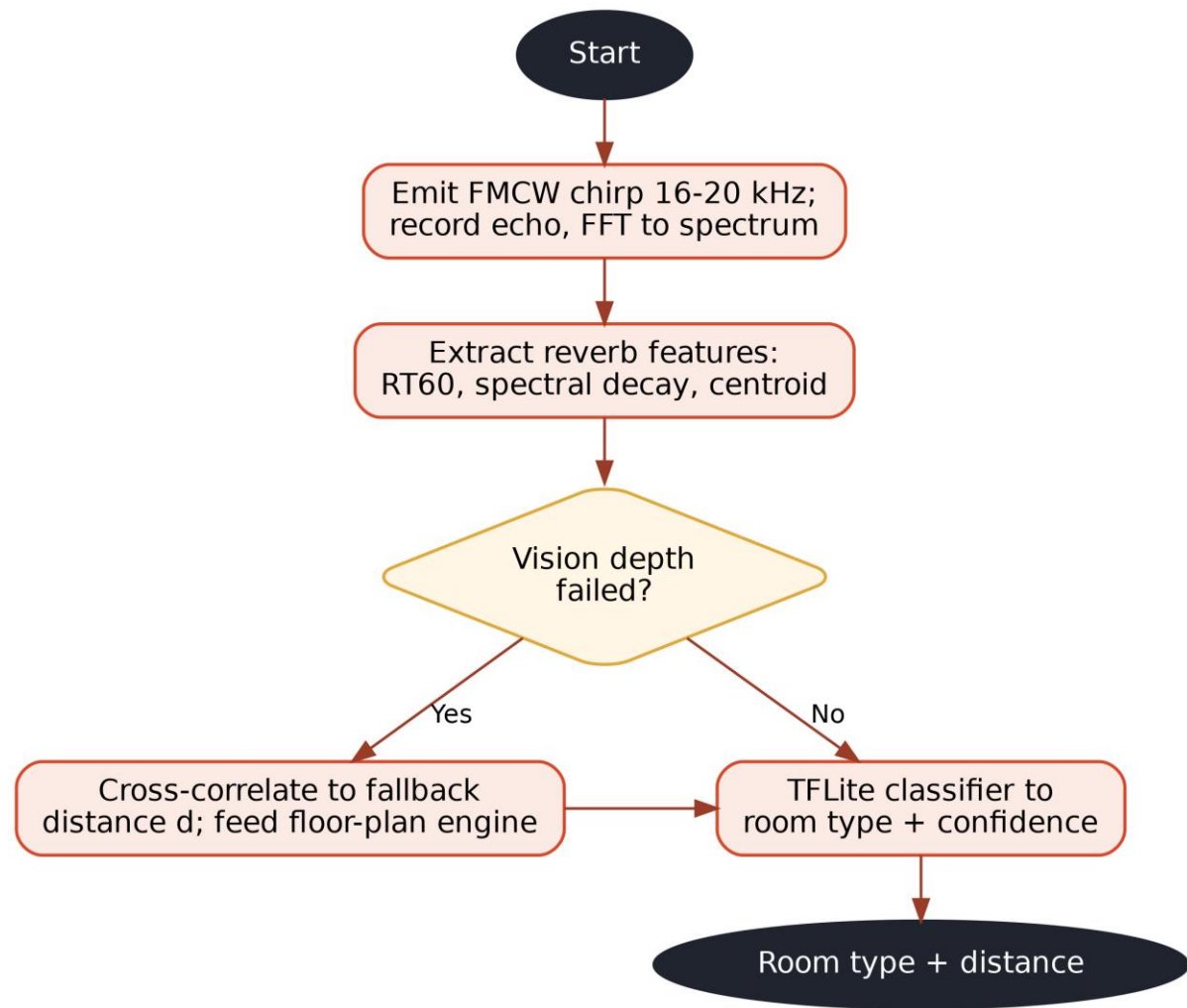


Figure 3.10: Flowchart — Acoustic Classification and Fallback Algorithm

3.11.3 Algorithm 3: A* Indoor Navigation

Step	Operation
Input	A loaded FloorPlan; start node s ; destination node d ; accessibility flag a .
Step 1	Build a NavGraph from the FloorPlan: walkable cells become nodes, adjacency becomes weighted edges, doors link spaces.
Step 2	Initialise open set with s ; set $g(s)=0$ and $f(s)=h(s,d)$ using straight-line (Euclidean) heuristic.
Step 3	WHILE open set not empty: pop node n with lowest f ; IF $n = d$, reconstruct and RETURN path.
Step 4	FOR each neighbour m of n : tentative_g = $g(n) + \text{cost}(n,m)$; IF better, update $g(m)$, $f(m)=g(m)+h(m,d)$, set parent.
Step 5	Convert the node path into turn-by-turn instructions and waypoints.
Step 6	IF $a = \text{true}$: render guidance as spoken instructions and haptic cues; ELSE render 2D map and AR overlay. Raise obstacle alerts from live ML Kit detections.
Output	A Route delivered through the appropriate visual or non-visual channel, with real-time obstacle alerts.

Table 3.12: Algorithm 3 — A* Indoor Navigation

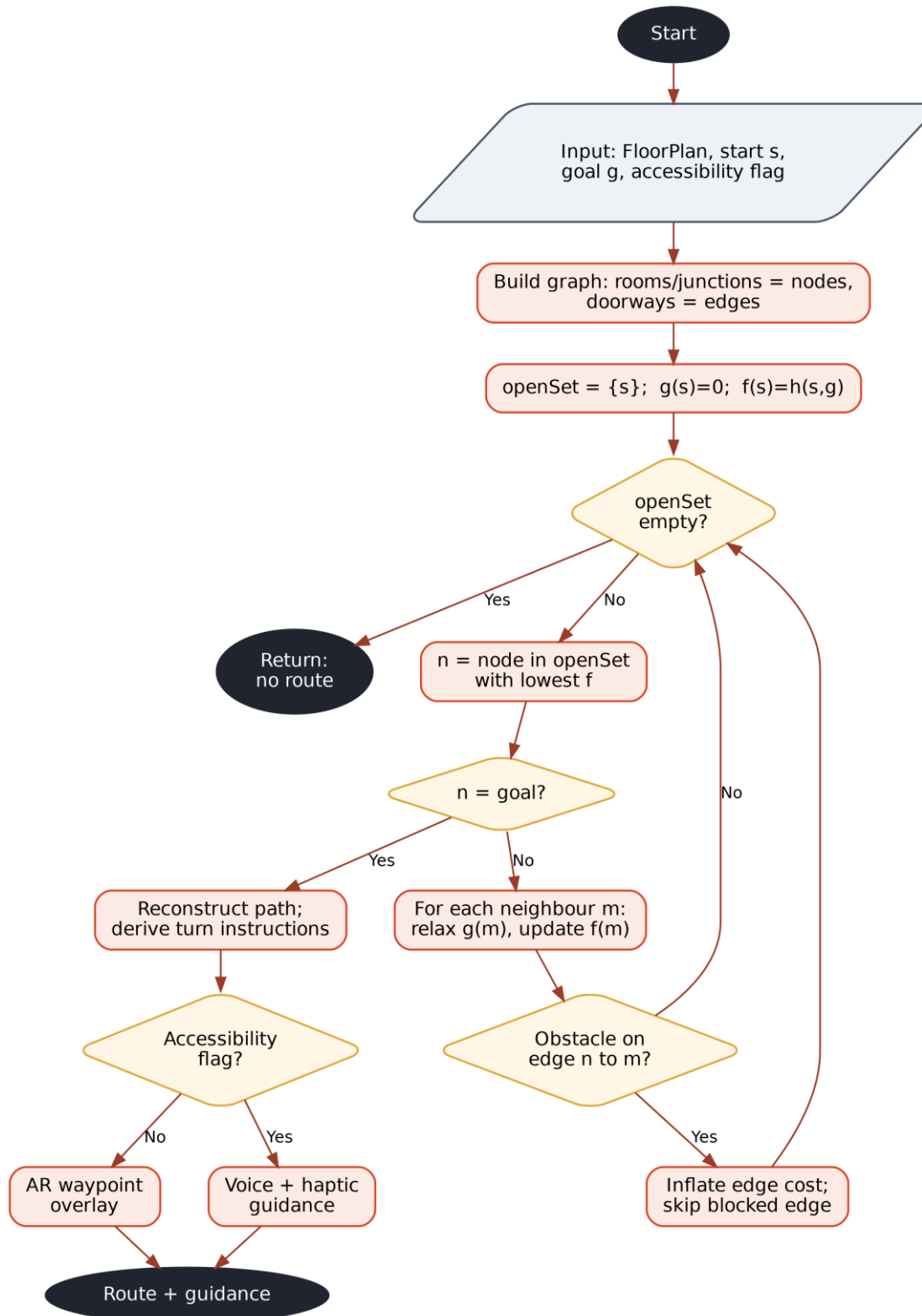


Figure 3.11: Flowchart — A* Indoor Navigation Algorithm

3.12 Project Methods to be Employed

EchoLocate adopts an Agile/iterative development approach. Unlike a plan-driven method, this suits a project with significant technical uncertainty across multiple novel sensing layers — vision-

based depth mapping, acoustic DSP, semantic detection, and accessibility navigation — where the behaviour of each modality on commodity hardware can only be established empirically. Structuring the work as iterations with measurable acceptance gates allows deficiencies in each sensing layer to be identified and corrected before full integration, and emphasises continuous benchmarking and documented trade-off decisions over the delivery of a feature-complete product. This directly manages the integration risk inherent in a multimodal system built and validated by a small team over a fixed 12-week timeline.

3.13 Software Process Model

EchoLocate follows an Agile/Iterative Development Model, delivering the system through five weekly-validated phases, each with a defined acceptance gate, over a 12-week timeline.

Phase	Focus	Key Activities	Acceptance Gate
1	Research and Foundation	Literature review; set up Flutter, ARCore, and ML Kit; validate camera/depth capture and FMCW chirp emission on physical devices.	Depth capture and chirp emission demonstrated on a physical device.
2	Computer-Vision Mapping Engine	Integrate ARCore Depth API; accumulate point cloud; implement RANSAC/Hough line fitting; integrate ML Kit object and text detection.	2D floor plan generated and benchmarked against a tape measure.
3	Acoustic DSP and Sensor Fusion	Implement FMCW chirp generation, FFT pipeline, and reverberation features; train the TFLite room classifier; implement acoustic fallback; fuse acoustic, ARCore, and IMU data.	Room-type classification and acoustic fallback operational.
4	Navigation, Crowdsourcing and Accessibility	Implement A* routing, turn-by-turn guidance, AR overlay, obstacle detection, Supabase repository, Drift persistence, 3D preview, voice mode, and PDF export.	Feature-complete application build.
5	Testing and Refinement	Map-accuracy, acoustic-accuracy, and navigation success-rate testing for sighted and accessibility-mode users; performance profiling; finalise report.	Validated, deployable build meeting accuracy and usability targets.

Table 3.13: EchoLocate Iterative Development Plan

This model was chosen because EchoLocate’s sensing layers carry genuine technical uncertainty that must be retired empirically and incrementally, unlike Waterfall, whose heavyweight upfront design assumes stable, fully understood requirements. Validating each modality at its acceptance gate before integration reduces the risk of compounding failures across the multimodal pipeline and keeps the limited project timeline focused on the highest-risk components first.

3.14 Project Design Considerations (Logical Designs)

3.14.1 User Interface Design

EchoLocate’s interface follows an accessibility-first, high-contrast design language built around a coral (#E8553A) primary colour on dark and light surfaces, with large touch targets, clear iconography, and text labels accompanying every colour-coded status so that guidance is never conveyed by colour alone. The design spans the full application flow — welcome and onboarding, authentication, permission requests, home and explore, active scanning with live quality feedback, map upload and community leaderboard, building detail, 2D turn-by-turn navigation,

3D floor-plan preview, the accessibility voice-navigation mode, and empty and offline states — produced as wireframes in a dedicated design tool and presented in Figures 3.12–3.31.



Figure 3.12: Welcome / Landing

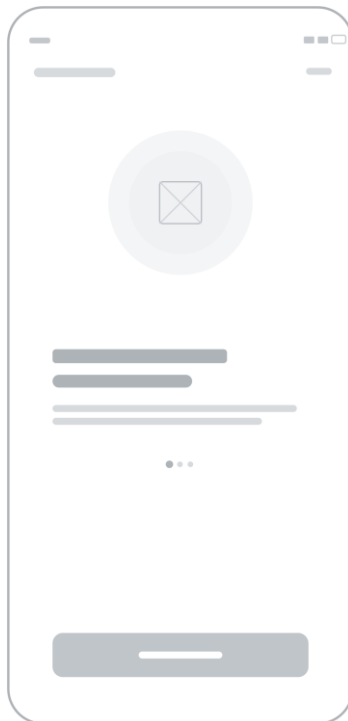


Figure 3.13: Onboarding — Accessibility

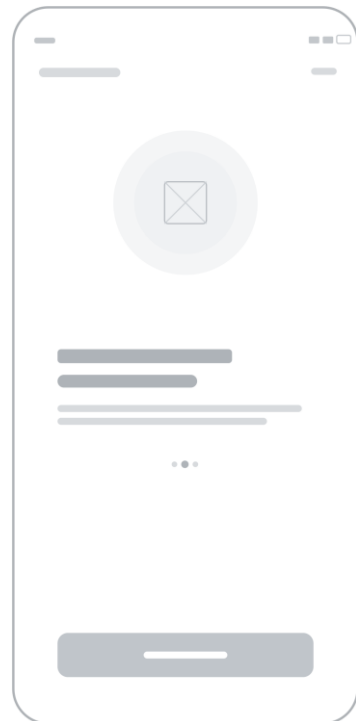


Figure 3.14: Onboarding — Scan a Space

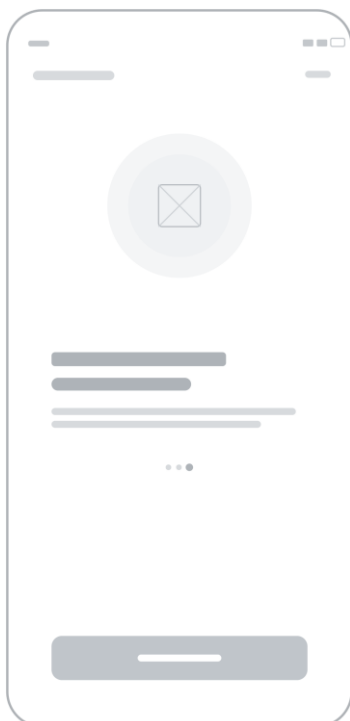


Figure 3.15: Onboarding — Navigate Indoors

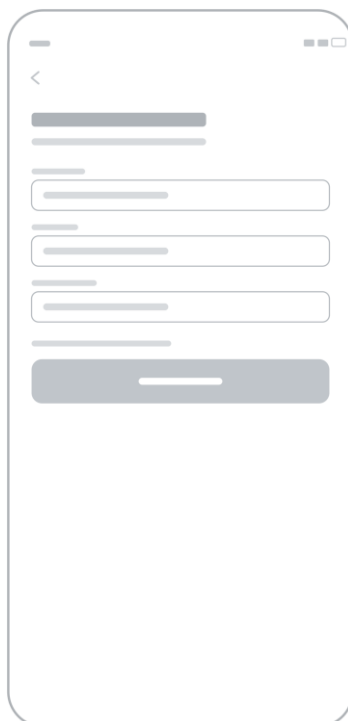


Figure 3.16: Create Account

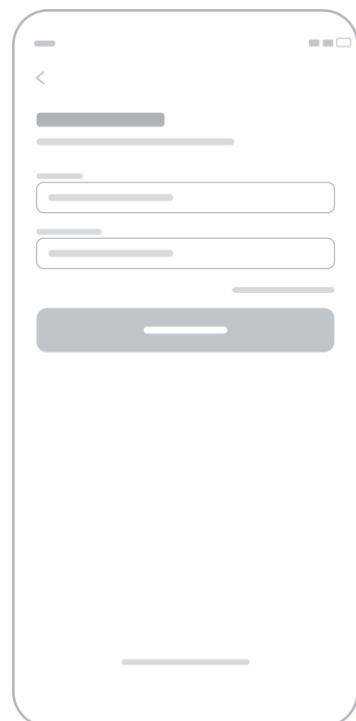


Figure 3.17: Sign In



Figure 3.18: Location Permission

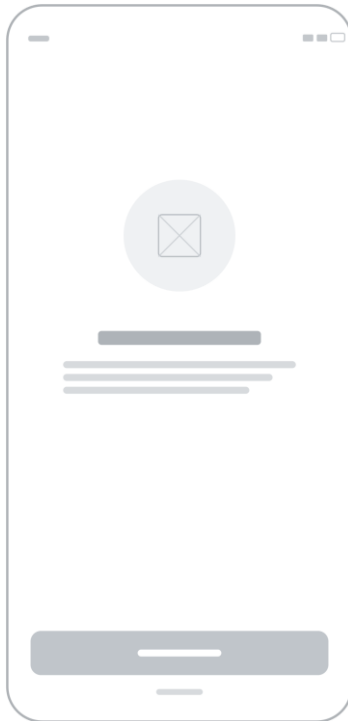


Figure 3.19: Camera Permission

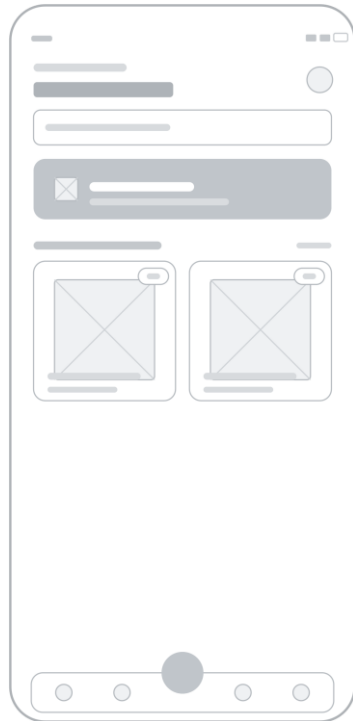


Figure 3.20: Home — "Where to?"



Figure 3.21: Explore Buildings



Figure 3.22: Active Scanning



Figure 3.23: Scan Quality Alert



Figure 3.24: Discard Scan Dialog



Figure 3.25: Map Uploaded & Leaderboard

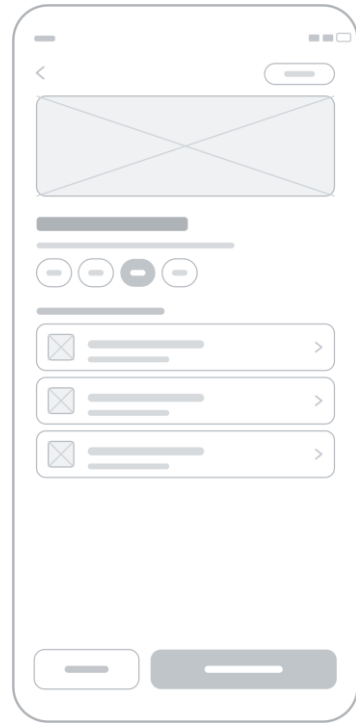


Figure 3.26: Building Detail

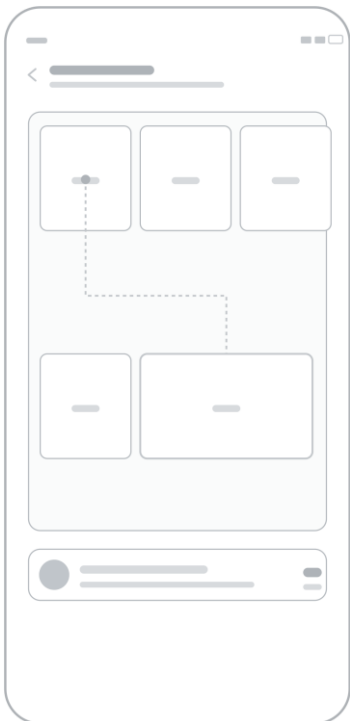


Figure 3.27: 2D Turn-by-Turn Navigation

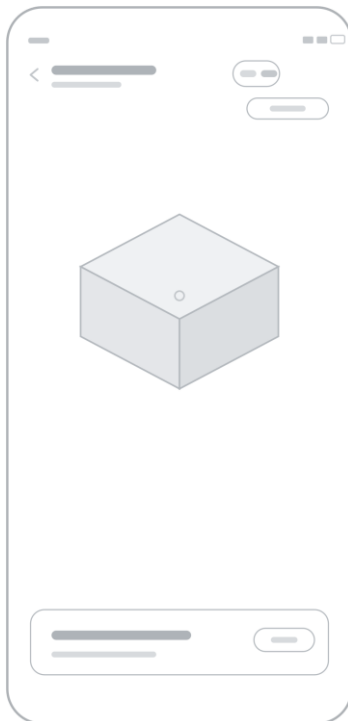


Figure 3.28: 3D Floor-Plan Preview

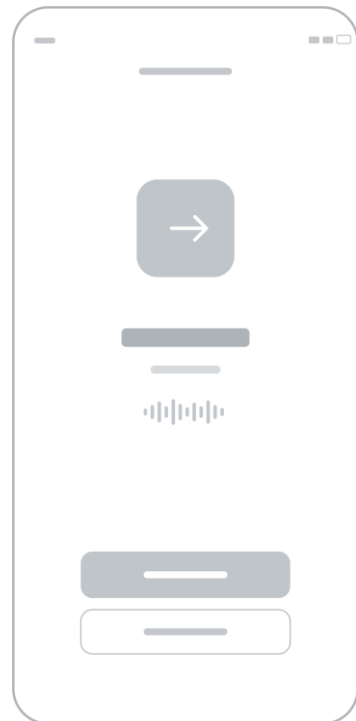
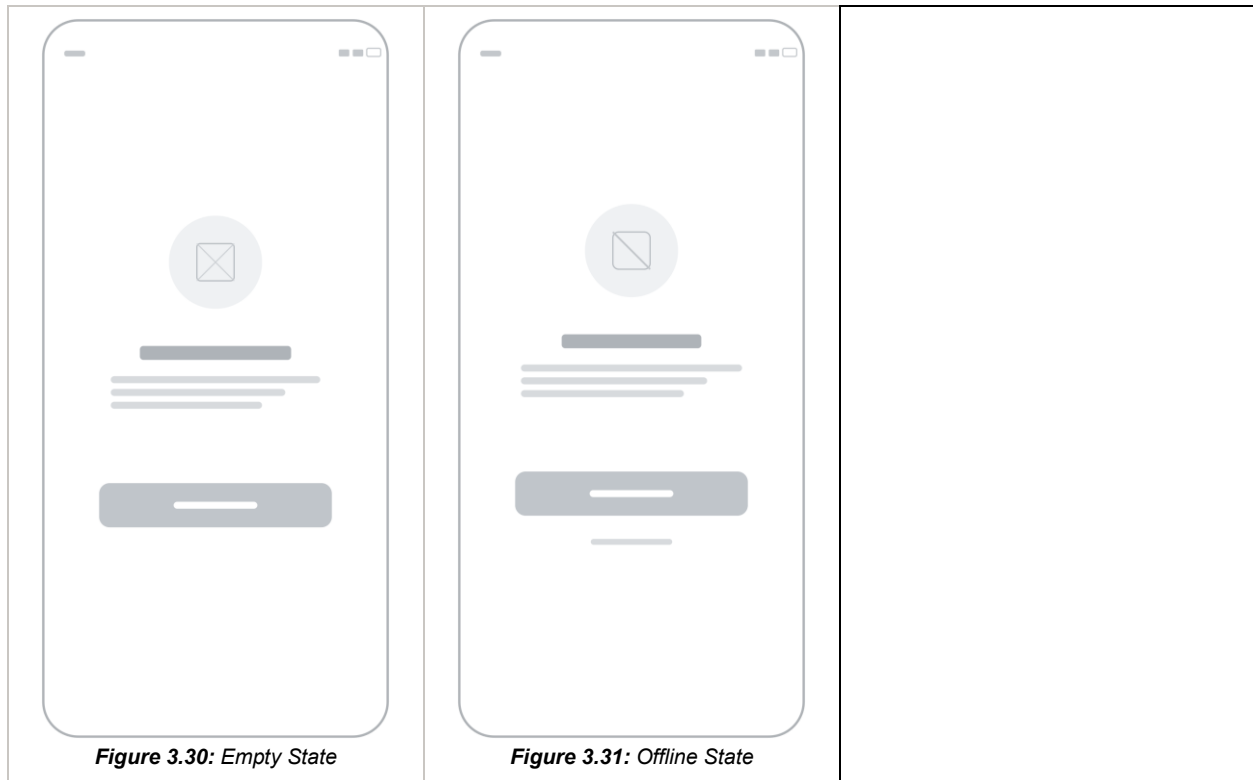


Figure 3.29: Accessibility Voice Navigation



Figures 3.12–3.31: EchoLocate User-Interface Wireframes

3.14.2 Database Design

EchoLocate uses a local-first Drift (SQLite) database for on-device persistence, with the Supabase cloud holding shared copies of uploaded maps. The local schema is normalised around a `floor_plans` table (plan metadata, building, quality indicator), a `scan_sessions` table linked to each plan, a `wall_segments` table and a `semantic_labels` table holding the geometry and detected features of a plan, and a `navigation_history` table recording past routes. Foreign keys with cascade deletion keep a plan and its dependent rows consistent, and the same logical structure is mirrored in the Supabase repository for community maps.

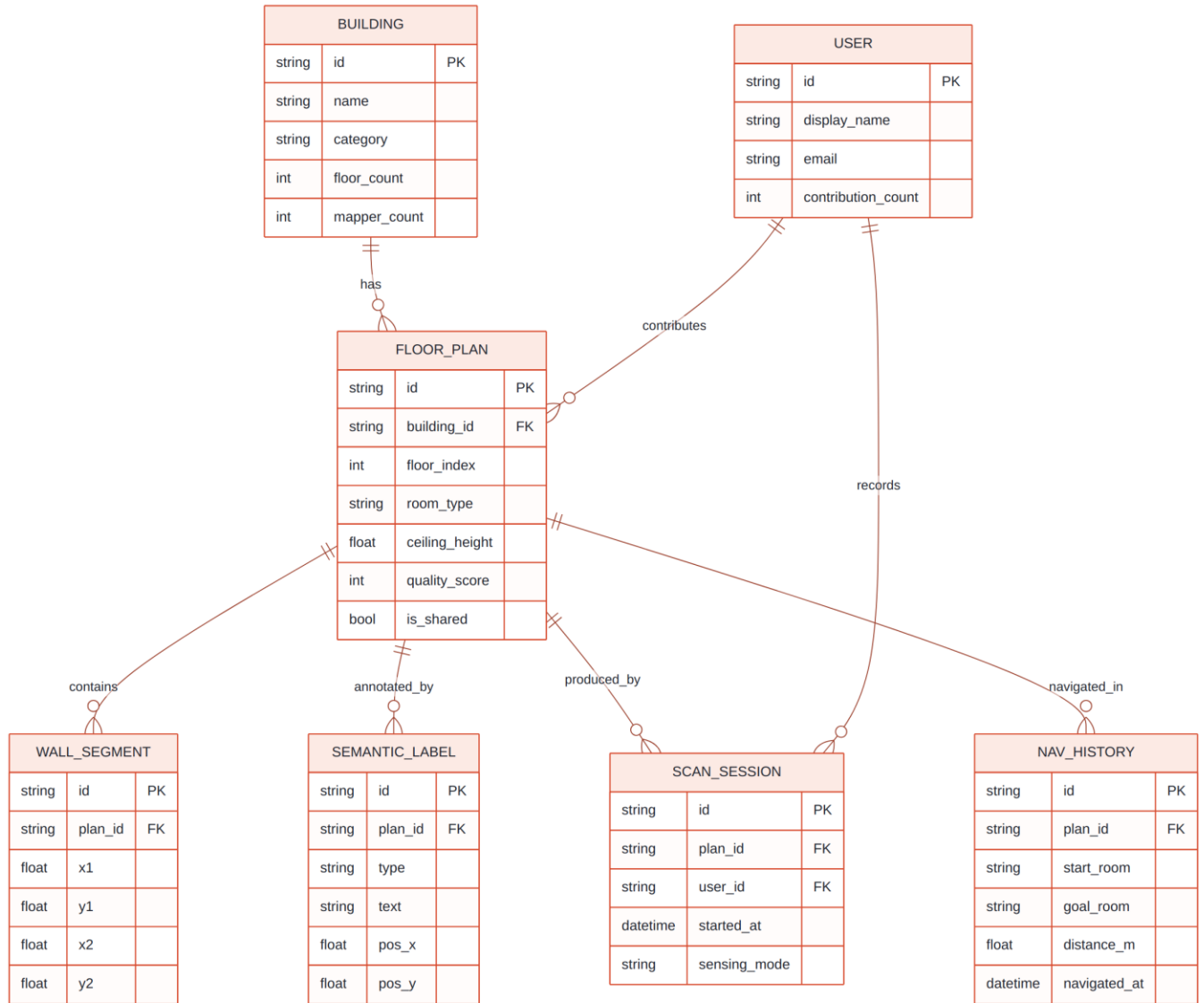


Figure 3.32: Entity-Relationship Diagram — EchoLocate Local Drift Database

3.15 Development Tools

Table 3.14 summarises the primary development tools and technologies used in EchoLocate, with reference to their role in the methodology described in this chapter.

Tool / Technology	Role in EchoLocate
Flutter (Dart)	Primary cross-platform framework; single codebase for the Android and iOS applications, all UI screens, and core application logic, with Riverpod for state management and Go Router for navigation.
ARCore Depth API	Vision-based depth-from-motion sensing; captures depth from the moving camera to build the point cloud used for floor-plan generation, accessed from Flutter via a native Android platform channel.
Google ML Kit	On-device object detection (doors, obstacles) and text recognition (room numbers, signs) during scanning and navigation, with no cloud processing.

Tool / Technology	Role in EchoLocate
TensorFlow Lite	Runs the trained acoustic room-type classifier on-device, mapping reverberation features to a predicted space type.
Dart FFT Pipeline	Performs the Fast Fourier Transform and reverberation feature extraction for room classification and the fallback distance estimate — the project's Digital Signal Processing component.
Drift (SQLite)	Local-first on-device database storing floor plans, scan sessions, and navigation history for fully offline operation.
Supabase	Cloud backend for the crowdsourced map repository; stores and serves community-uploaded floor plans, contributor records, and map-quality ratings.
flutter_tts & speech_to_text	Voice interface delivering spoken turn-by-turn guidance and accepting voice commands for the accessibility mode.
A* Routing Engine (custom Dart)	Computes the shortest navigable path through a loaded floor plan to drive turn-by-turn navigation.
flutter_gl	Renders the extruded 3D preview of the generated floor plan.
pdf (Dart package)	Generates shareable PDF floor-plan reports from captured spatial data.

Table 3.14: *EchoLocate Development Tools*

Chapter Three References

- Duda, R. O., & Hart, P. E. (1972). Use of the Hough transformation to detect lines and curves in pictures. *Communications of the ACM*, 15(1), 11–15.
- Fischler, M. A., & Bolles, R. C. (1981). Random sample consensus: A paradigm for model fitting with applications to image analysis and automated cartography. *Communications of the ACM*, 24(6), 381–395.
- Hart, P. E., Nilsson, N. J., & Raphael, B. (1968). A formal basis for the heuristic determination of minimum cost paths. *IEEE Transactions on Systems Science and Cybernetics*, 4(2), 100–107.
- Nielsen, J. (1994). *Usability Engineering*. Morgan Kaufmann.
- Republic of Ghana. (2012). *Data Protection Act, 2012 (Act 843)*. Accra: Ghana Publishing Company.
- World Wide Web Consortium (W3C). (2018). *Web Content Accessibility Guidelines (WCAG) 2.1*. W3C Recommendation.
- Google. (2024). *ARCore Depth API documentation*. Google Developers.